



PALESTINE POLYTECHNIC UNIVERSITY

Faculty of Information Technology and Computer Engineering

Department of Computer Engineering

INFORMATION SECURITY PROJECT

Secura :Secure File Protection

Anas Hroub - 221109

Iyad Maree - 221026

Supervised by
Dr. Radwan Tahboub

June 23, 2026

Abstract

Protecting sensitive files from unauthorized access has become an important requirement for many computer users. While traditional file protection methods often rely on encryption, they may be difficult for non-technical users or introduce additional system overhead.

This project presents **Secura**, a lightweight Windows desktop application that provides file protection through a dual-layer security approach. The system uses Windows Access Control Lists (ACLs) to quickly lock files and restrict access, while also offering AES-256 encryption for enhanced protection of sensitive data.

The application is developed using Python and Tkinter, with SQLite used for local data storage. User passwords are secured using the PBKDF2 hashing algorithm to prevent plain-text storage.

The results show that Secura provides an effective balance between security, usability, and performance, making it a practical solution for local file protection on Windows systems.

الملخص

مع التزايد المستمر في حجم البيانات الرقمية المخزنة على الحواسيب الشخصية، أصبحت حماية الملفات الحساسة من الوصول غير المصرح به من المتطلبات الأساسية للمستخدمين. تعتمد العديد من حلول حماية الملفات التقليدية على التشفير فقط، مما قد يؤدي إلى تعقيد الاستخدام أو زيادة استهلاك موارد النظام عند التعامل مع الملفات بشكل يومي. يقدم هذا المشروع نظام Secura، وهو تطبيق مكتبي يعمل على نظام Windows ويوفر آلية متكاملة لحماية الملفات والمجلدات من خلال الجمع بين التحكم في الوصول والتشفير. يعتمد النظام على طريقتين رئيسيتين للحماية، هما القفل السريع باستخدام صلاحيات NTFS Access Control Lists (ACLs) والتشفير العميق باستخدام خوارزمية AES-256-GCM، مما يمنح المستخدم المرونة في اختيار مستوى الحماية المناسب حسب طبيعة البيانات وأهميتها. يتضمن النظام مجموعة من الميزات الأمنية المتقدمة، مثل كلمة المرور الرئيسية، وتشفير البيانات الوصفية المخزنة في قاعدة البيانات، وسجلات أمنية تدعم اكتشاف العبث، ومفتاح استرداد، وخزينة استرداد مشفرة، وإعادة تعيين كلمة المرور باستخدام رمز تحقق OTP عبر البريد الإلكتروني، بالإضافة إلى النسخ الاحتياطي السحابي والتعافي من الكوارث. كما يدعم النظام تنسيق ملفات مشفر خاص يتيح التحقق من كلمة المرور واستعادة الملفات حتى في بعض حالات فقدان بيانات قاعدة البيانات. تم تطوير النظام باستخدام لغة Python وقاعدة بيانات SQLite، وتم تقييمه من خلال مجموعة من الاختبارات الوظيفية والأمنية واختبارات الأداء. أظهرت النتائج أن النظام يوفر توازناً فعالاً بين الأمان وسهولة الاستخدام والأداء، حيث يوفر القفل السريع حماية فورية للملفات والمجلدات، بينما يوفر التشفير العميق مستوى مرتفعاً من السرية وسلامة البيانات للملفات الحساسة.

0.1 Acknowledgment

We would like to express our sincere gratitude to our supervisor, Dr. Eng. Radwan Tahboub, for his valuable guidance, continuous support, and constructive feedback throughout the development of this project. His expertise and encouragement played a significant role in improving the quality of our work and helping us achieve the project objectives.

We would also like to extend our appreciation to Dr. Eng. Mousa Farajallah for his supervision and guidance during the early stages of the project. His recommendations and insights contributed greatly to establishing a strong foundation for the work.

Our sincere thanks are extended to Palestine Polytechnic University, the Faculty of Information Technology and Computer Engineering, for providing the academic environment, knowledge, and resources that supported the completion of this project.

Finally, we would like to thank the examination committee members for dedicating their time and effort to review and evaluate our work. Their comments and suggestions are highly appreciated and will contribute to the improvement of this project.

Thank you all for your support and contribution to the successful completion of this work.

Contents

0.1	Acknowledgment	1
1	Introduction	1
1.1	Overview	1
1.2	Background and Context	1
1.3	Problem Statement	2
1.4	Project Objectives	2
1.5	Scope and Limitations	3
1.6	Report Organization	4
1.7	Methodologies	5
2	Literature Review	6
2.1	Overview of the Domain	6
2.1.1	Cryptographic Protection vs. Access Control	6
2.1.2	Windows NTFS and ACLs	7
2.2	Existing Methods and Related Work	7
2.2.1	Encryption and File Hiding Applications	7
2.2.2	Native Access Control List (ACL) Management	8
2.2.3	Gap Analysis	8
3	System Analysis and Design	9
3.1	System Requirements	9
3.1.1	Functional Requirements	9
3.1.2	Non-Functional Requirements	12
3.2	System Architecture	13
3.3	Use Case Diagram	14
3.4	Data Flow Diagram (DFD)	15
3.5	Security Design and Threat Model	15
4	Implementation and Methodology	18
4.1	Development Environment	18
4.2	Implementation Steps	19
4.3	Algorithms and Models Used	20
4.4	Security Mechanisms	28
4.5	User Interface and System Screens	29
5	Testing, Evaluation, and Results	30
5.1	Testing Strategy	30

5.2	Experimental Setup	30
5.3	Performance Metrics	31
5.4	Results and Analysis	31
5.4.1	Execution Time Comparison (Fast Lock vs. Deep Encryption) . . .	31
5.4.2	Cryptographic Engine Performance	32
5.5	Security Evaluation	33
6	Conclusion and Future Work	34
6.1	Summary of Work	34
6.2	Conclusions	34
6.3	Future Work	35
A	Source Code	38
A.1	Master Password Creation and Authentication	38
A.2	AES-256-GCM File Encryption Algorithm	40
A.3	AES-256-GCM File Decryption Algorithm	42
A.4	NTFS ACL Fast Lock and Unlock Implementation	43
A.5	Recovery Key and Recovery Vault Mechanism	45
A.6	Google Drive Backup and Disaster Recovery Module	47
B	User Guide	49
B.1	First-Time Setup	50
B.2	Recovery Key and QR Code Screen	51
B.3	Managing Protected Files	52
B.4	File Protection Operations	53
B.5	Security Logs	54
B.6	Cloud Backup, Disaster Recovery and Repair Files	55
B.7	Settings and Maintenance	58
B.8	Help and User Manual	59

List of Figures

1.1	Scope and Limitations	4
3.1	Proposed System Architecture	13
3.2	Use Case Diagram	14
3.3	Data Flow Diagram	15
4.1	Cloud Backup and Auto-Sync	27
4.5.1	Results and Analysis	32
4.5.1	First-Time Setup	50

4.5.2 Login Interface	51
4.5.3 Main Dashboard	52
4.5.4 Secure New Item Interface	53
4.5.5 Security Logs Interface	54
4.5.6 Cloud Backup Interface	55
4.5.7 Disaster Recovery Interface	56
4.5.8 Repair Files	57
4.5.9 Settings Interface	58
4.5.10 Help and Manual Interface	59

List of Tables

5.1 Execution Time Comparison	32
5.2 Cryptographic Performance Metrics	33

Chapter 1

Introduction

1.1 Overview

Information security has become increasingly important as users rely on personal computers to store private documents, academic materials, work files, and other sensitive information. Unauthorized access, data theft, accidental exposure, and misuse of shared devices remain common threats that require effective file protection mechanisms [1].

Existing file protection solutions often focus on a single approach. Encryption-based tools provide strong confidentiality but may be slow or complex for everyday use, while file hiding and access restriction methods are generally easier to use but offer limited protection against advanced attackers. Consequently, users often face a trade-off between security, performance, and usability.

To address this challenge, the Secura project introduces a Windows desktop application that combines fast file locking using NTFS Access Control Lists (ACLs) with deep file encryption based on AES-256. In addition, the system provides secure authentication, recovery mechanisms, security logging, cloud backup, and disaster recovery features through a unified and user-friendly interface.

The system allows users to select the appropriate protection level according to the sensitivity of their data while maintaining simplicity and efficiency. Furthermore, the custom .secura file format supports password validation and recovery operations, while protected logs help detect tampering and incomplete operations. As a result, Secura provides both preventive security and reliable recovery capabilities for local file protection on Windows systems.

1.2 Background and Context

In recent years, the amount of sensitive data stored on personal computers has increased significantly. Users commonly store identity documents, financial records, private photos, academic work, and business files on their devices, many of which are shared with others, increasing the risk of unauthorized access [1].

File protection methods generally fall into two categories: cryptographic protection and access control. Encryption secures file contents using secret keys and remains effective even if files are copied or accessed outside the operating system, although it may impact performance and requires key management. Access control relies on operating system permis-

sions, such as NTFS Access Control Lists (ACLs), to restrict file operations. This approach is fast and practical but can be bypassed by administrators or through direct disk access.

Secura combines both approaches in a dual-layer security model. It uses ACL-based locking for fast protection and AES-256 encryption for strong confidentiality. The system also includes recovery and forensic features such as recovery keys, encrypted recovery storage, secure database encryption, tamper-evident logs, and recovery scanning, making it more resilient and suitable for real-world security environments.

1.3 Problem Statement

Many users need to protect files on Windows devices, especially when those devices are shared or physically accessible by others. However, existing solutions often fail to provide a balanced approach. Encryption-only applications may be too complex or resource-consuming for simple daily use, while file hiding tools and manual permission changes provide weak or error-prone protection. In addition, many tools do not provide reliable recovery mechanisms when a password is changed, a database record is lost, an encryption process is interrupted, or local data becomes unavailable due to device or database failure. A major challenge is that file protection must address different threat levels. Casual local users may be stopped using operating-system-level restrictions, but advanced attackers who bypass Windows permissions require cryptographic protection. At the same time, the system must remain usable for non expert users and must reduce the risk of accidental file loss, corruption, permanent lockout, or loss of access to protected data after unexpected failures. Therefore, the main problem can be defined as follows: How can we design and implement a secure, user-friendly Windows desktop application that provides flexible file protection through both fast ACL-based locking and strong authenticated encryption, while also supporting password recovery, tamper detection, safe file repair mechanisms, and backup-based disaster recovery? The Secura system addresses this problem by integrating NTFS ACL locking, AES-256 encryption, secure master password storage, recovery-key-based rescue, email OTP verification, encrypted database metadata, tamper-evident logging, automated recovery scanning, and cloud backup and disaster recovery into one unified application.

1.4 Project Objectives

The main objective of the Secura project is to develop a secure and user-friendly Windows desktop application that provides effective file protection and recovery mechanisms. The system aims to combine fast file locking using NTFS Access Control Lists (ACLs) with strong AES-256-GCM encryption to address different security requirements. It also seeks to enhance authentication and recovery through master password management, email OTP verification, and recovery key support. In addition, the project aims to maintain secure activity logs, provide file repair and recovery capabilities, and support cloud backup and disaster recovery through Google Drive integration. Ultimately, Secura is designed to achieve a balance between security, performance, usability, and data availability while remaining suitable for everyday use.

1.5 Scope and Limitations

Scope

This project focuses on building a local Windows desktop application that protects files and folders using both operating-system-level access control and cryptographic encryption. The system provides fast file and folder locking through NTFS ACL DENY rules [2], as well as deep file encryption using AES-256-GCM [3]. It also supports a custom .secura encrypted file format with password validation data and a fast encryption mode that partially encrypts the first 4096 bytes of a file.

The application includes master password creation, verification, and password policy enforcement. It uses an SQLite database [4] to store encrypted metadata, authentication information, recovery information, and logs, while applying database string encryption to protect file names, paths, and sensitive metadata.

The system also provides recovery and repair features, including recovery key generation and verification, recovery vault encryption, email OTP support for password reset, and a Repair All Files feature for re-encrypting files under a new master password. In addition, it includes a recovery scanner for detecting damaged or orphaned protected files.

Finally, the project includes tamper-evident security logs, encrypted shadow records, automatic logout after inactivity, and PDF log export. Overall, the scope of this project is limited to local file and folder protection on Windows without cloud-based file management.

Limitations

Despite its security features, the system has several limitations. The application is designed specifically for Windows operating systems, and the fast ACL locking feature requires the NTFS file system. In addition, administrator privileges are required because ACL manipulation depends on system-level permission changes. The system also uses one global master password instead of separate passwords for each protected file. While ACL-only protection provides fast access control, it can still be bypassed by a highly privileged administrator or by accessing the disk outside Windows. Therefore, full cryptographic confidentiality is guaranteed only for files protected using AES-256-GCM encryption.

The system also depends on some external and user-managed factors. Email OTP requires a configured SMTP account and internet connectivity, while the recovery key must be stored safely by the user. If both the master password and the recovery key are lost, access to encrypted files may not be recoverable. Finally, multi-user account management are outside the current scope of this project.

Figure 1.1 presents the scope and limitations of the Secura system.

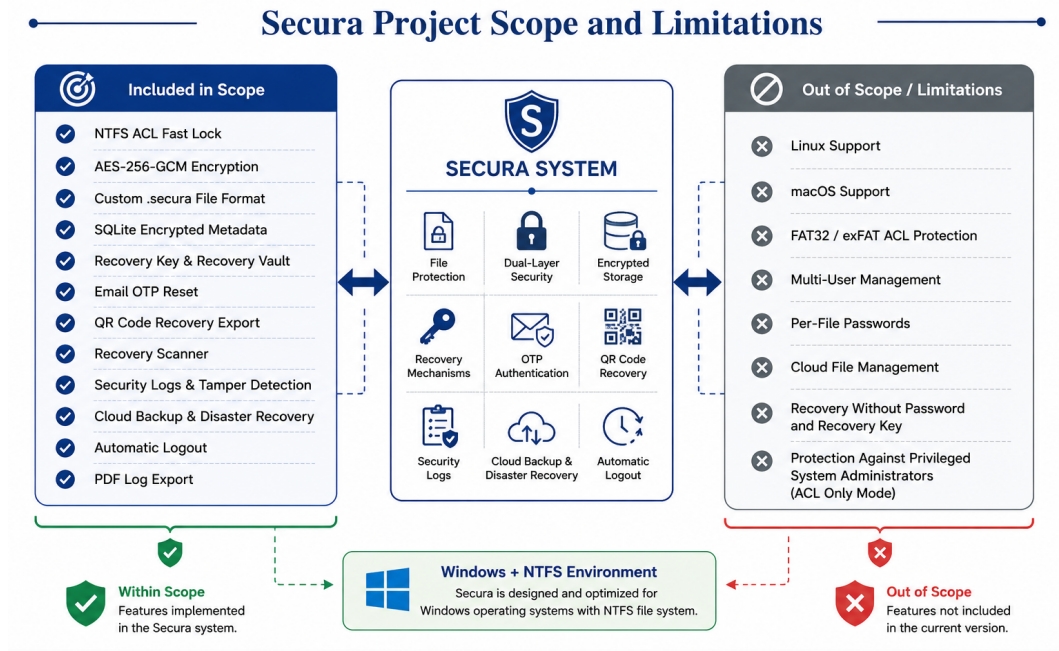


Figure 1.1: Scope and Limitations

1.6 Report Organization

This report is organized into six main chapters and two appendices. Chapter 1 introduces the project background, problem statement, objectives, scope, limitations, and methodology. Chapter 2 presents the literature review by discussing file protection approaches, cryptographic protection, access control, existing tools, and the gap that motivates the development of Secura. Chapter 3 focuses on system analysis and design, including system requirements, architecture, use case diagram, data flow diagram, and security threat model.

Chapter 4 explains the implementation and methodology of the project, including the development environment, implementation steps, main algorithms, cryptographic mechanisms, database design, and system interface. Chapter 5 presents the testing, evaluation, and results by describing the testing strategy, experimental setup, performance metrics, results, and security evaluation. Finally, Chapter 6 summarizes the project outcomes, presents the main conclusions, and suggests possible future enhancements. Appendix A includes selected source code snippets and implementation details, while Appendix B provides installation instructions and usage steps for the Secura application.

1.7 Methodologies

The development of the **Secura** project follows a structured and iterative process consisting of analysis, design, implementation, testing, and refinement.

- **Analysis:** In this phase, the problem domain was studied by analyzing common file protection techniques, including encryption software, file hiding tools, and Windows ACL permission management. The main requirements were identified based on the need for a secure, flexible, and user-friendly file protection system.
- **Design:** During the design phase, the system architecture was planned by separating the application into several modules: graphical user interface, authentication manager, ACL manager, encryption module, database manager, recovery manager, and log integrity manager. The custom .secura file format, database schema, password recovery workflow, and security log design were also defined.
- **Implementation:** The system was implemented using Python [5], CustomTkinter [6], SQLite [4], pywin32, and cryptographic libraries [7]. The application integrates AES-256-GCM encryption, PBKDF2 key derivation, NTFS ACL manipulation through Windows tools, encrypted database storage, email OTP, recovery key management, and tamper evident logging
- **Testing:** The testing phase included functional testing, security testing, recovery testing, usability testing, and performance testing. The system was tested against common scenarios such as successful encryption and decryption, wrong password attempts, ACL bypass attempts, password reset, file repair, corrupted .secura files, missing database records, and log tampering.
- **Refinement:** After testing, several improvements were applied to increase stability and usability. These included better error handling, safer database lock/unlock operations, improved GUI state management, idle session timeout, protection against duplicate file records, and recovery mechanisms for interrupted operations.

Chapter 2

Literature Review

2.1 Overview of the Domain

File protection is an important part of information security because it focuses on maintaining the confidentiality, integrity, and availability of sensitive user data [8]. On personal computers, file protection is especially important because files may be exposed to local users, malware, physical theft, unauthorized access through shared devices, or unexpected data loss caused by system or storage failure. The most common techniques used to protect local files include encryption, access control, file hiding, password-based vaults, secure deletion, backup, and recovery mechanisms. Each technique provides a different level of security. Encryption provides strong confidentiality by making file contents unreadable without the correct key. Access control prevents unauthorized users from opening or modifying files through operating-system permissions [9]. File hiding only reduces visibility but does not provide real security. Backup and disaster recovery techniques help maintain availability by allowing data to be restored when local records or protected system files become unavailable. The Secura system belongs to the domain of local file protection and secure recovery. It combines access control and cryptographic protection while adding forensic features such as tamper detection, recovery scanning, secure repair after password changes, and cloud-based recovery support.

2.1.1 Cryptographic Protection vs. Access Control

Cryptographic protection converts plaintext into ciphertext using an encryption algorithm and a secret key. When implemented correctly, it protects data even if an attacker obtains a copy of the file. Secura uses AES-256-GCM [3], which provides both confidentiality and integrity through authenticated encryption [10]. This is especially important for encrypted files and backup data stored outside the local device.

Access control relies on operating system permissions. In Windows, NTFS Access Control Lists (ACLs) [2] define which users can access files and what actions they may perform. ACL-based protection is fast and does not modify file contents, making it suitable for daily use. However, it may be bypassed by administrators or through direct disk access. Therefore, encryption protects the data itself, while access control restricts access within the operating system. Secura combines both approaches to provide layered protection.

2.1.2 Windows NTFS and ACLs

The Windows NTFS file system supports Access Control Lists (ACLs) that allow or deny permissions for users and groups. Secura uses this feature to implement fast file locking by applying restrictive DENY rules to the Everyone group through administrative privileges. This prevents normal local access to protected files.

Before modifying permissions, the system stores backup information so that permissions can be restored during unlocking, repair, or recovery operations. Although ACL management can be difficult for non-technical users, Secura simplifies the process through a graphical interface that automates permission changes and reduces configuration errors.

2.2 Existing Methods and Related Work

Existing file protection solutions can be grouped into four main categories: encryption-based applications, file hiding tools, native access control tools, and backup or recovery-based solutions. Encryption-based applications provide strong confidentiality by encrypting file contents. Tools such as encrypted containers and vault applications [11] are suitable for high-security scenarios. However, they may be inconvenient for users who only need quick protection for everyday files. They may also require additional steps such as mounting volumes, managing keys, or moving files into special containers. File hiding tools attempt to protect files by making them invisible in the file explorer or by changing file attributes. This method is weak because hidden files can usually be revealed through Windows settings or command-line commands. File hiding does not protect the actual content of the file.

Native Windows access control tools allow users to manually edit file permissions. While this approach can be effective, the user interface is complex and requires technical understanding of inheritance, ownership, users, groups, and permission precedence. Incorrect configuration may lock users out of their own files or create system instability. Backup and recovery solutions focus on restoring data after accidental deletion, storage failure, or system damage. However, many backup tools are separated from the file protection workflow and may not provide encrypted application-level recovery for protected files and security metadata. Secura improves on these approaches by combining quick ACL protection, strong authenticated encryption, secure password handling, recovery mechanisms, forensic log monitoring, and cloud backup and disaster recovery support in one application.

2.2.1 Encryption and File Hiding Applications

Encryption applications are effective against advanced threats because they protect the file content itself. Even if an attacker copies the encrypted file or removes the hard drive, the file remains unreadable without the correct password-derived key. However, encryption can be slower for large files and may be more complicated for casual users. File hiding applications are easier to use but provide limited security. They may change folder attributes, rename folders, or hide files from normal explorer views. These methods can stop casual browsing but cannot protect data from users who know how to reveal hidden files or access the file system directly. Secura avoids relying on file hiding alone. When folder obfuscation techniques are used, they are combined with ACL DENY rules and, when needed, encryption. This layered approach improves the security level beyond simple hiding and keeps sensitive data protected even when backup or recovery operations are involved.

2.2.2 Native Access Control List (ACL) Management

Windows provides built-in tools for ACL management through the file properties security tab and command-line tools such as `icacls` [12]. These tools are powerful but not designed for ordinary users who simply want to lock or unlock files safely. Manual ACL management introduces several risks. A user may remove important permissions, block their own account, fail to restore inheritance correctly, or apply permissions to the wrong object. In addition, the process requires many steps and may not provide clear feedback. Secura uses ACLs internally but hides their complexity from the user. The application applies predefined safe permission rules, stores backup data, logs operations, and provides recovery support. This makes NTFS access control easier and safer to use.

2.2.3 Gap Analysis

The existing ecosystem shows a gap between usability, security, and recovery continuity. Encryption tools provide strong protection, but they may be heavy or inconvenient for everyday use. File hiding tools are simple, but they usually provide weak protection. Native Windows ACL tools are powerful, but they can be complex and error-prone for regular users. Backup solutions can help restore lost data, but they are often separate from the protection system and may not preserve encrypted metadata, protected file states, logs, or recovery information in a secure and integrated way [13].

In addition, few existing solutions combine fast locking, deep encryption, password recovery, metadata protection, tamper-evident logs, automated repair, recovery scanning, and cloud-based disaster recovery in one unified system. Secura fills this gap by providing a Windows desktop application that offers fast ACL-based locking for everyday protection, AES-256-GCM encryption [3] for high-sensitivity files, a custom self-validating .secura encrypted file format, secure master password hashing and policy enforcement, encrypted database metadata, recovery key support, email OTP password reset, repair mechanisms for files encrypted under an old password, tamper-evident logs with shadow records, recovery scanning for damaged or orphaned protected files, and cloud backup support for restoring protected system data after local failure. This makes Secura more than a traditional file locker; it becomes a secure local file protection, forensic recovery, and disaster recovery system designed for Windows environments.

Chapter 3

System Analysis and Design

3.1 System Requirements

The system analysis phase was conducted based on the findings presented in Chapter 1 and Chapter 2. In Chapter 1, the project problem, objectives, scope, limitations, and user needs were identified, providing the foundation for defining the required system capabilities. In Chapter 2, existing file protection approaches, including cryptographic protection, access control mechanisms, recovery solutions, and related tools, were reviewed and analyzed. The gap analysis highlighted the lack of integrated solutions that combine fast access control, strong encryption, recovery mechanisms, tamper detection, and disaster recovery in a single user-friendly application. Based on these findings, the system requirements, architecture, security model, and design components presented in this chapter were derived and specified.

3.1.1 Functional Requirements

System Functional Requirements

- The system shall allow the user to create a master password during the first launch of the application.
- The system shall enforce a password policy requiring at least 12 characters, including uppercase letters, lowercase letters, digits, and symbols.
- The system shall require successful authentication before opening the main dashboard.
- The system shall verify Windows Administrator privileges before performing ACL-based operations.
- The system shall allow users to protect files and folders using either Fast Lock or Deep Encryption.
- The system shall store original NTFS ACL permissions before applying restrictive DENY rules.
- The system shall allow users to unlock protected files and folders by restoring original permissions.
- The system shall support AES-256-GCM encryption for sensitive files.

- The system shall derive encryption keys using PBKDF2-HMAC-SHA256 and a random salt.
- The system shall securely remove the original plaintext file after successful encryption.
- The system shall support decryption of protected `.secura` files after successful authentication.
- The system shall validate the encrypted file header before decryption.
- The system shall use a custom `.secura` file format containing a magic value, salt, nonce values, validation ciphertext, and authentication tags.
- The system shall support a fast encryption mode that encrypts only the first 4096 bytes of a file.
- The system shall automatically detect the encryption mode during decryption.
- The system shall provide a unified interface displaying protected items, including names, paths, protection methods, status, and timestamps.
- The system shall allow users to unlock, decrypt, and repair protected files.
- The system shall use SQLite [4] as the local database for authentication data, protected file metadata, recovery information, and logs.
- The system shall support cloud backup and disaster recovery.
- The system shall allow users to connect a personal cloud account through secure authorization [14].
- The system shall upload and restore encrypted backup copies of database and recovery-related information.
- The system shall provide a 24-character recovery key.
- The system shall provide QR-code export for the recovery key.
- The system shall support password reset through email OTP verification.
- The system shall use a six-digit OTP delivered through SMTP over STARTTLS.
- The system shall provide a Repair All Files feature to re-encrypt files after a master-password change.
- The system shall allow the old password to be entered directly or recovered through the recovery vault.
- The system shall record login attempts, protection operations, password resets, recovery actions, repairs, backups, and restore operations.
- The system shall include a recovery and health scanner.

- The scanner shall detect broken database links, missing files, orphaned .secura files, damaged encrypted files, and interrupted temporary files.
- The system shall support cloud-based vault recovery.
- The system shall automatically lock inactive sessions.
- The system shall support PDF export of security logs.

Security Functional Requirements

- The system shall support confidentiality, integrity, authentication, and recoverability.
- Passwords shall never be stored in plaintext.
- Passwords shall be protected using salted PBKDF2-HMAC-SHA256 hashes [15].
- Session-sensitive data shall remain in memory only while the user is authenticated.
- Sensitive database metadata, including file names, file paths, and log details, shall be encrypted at rest.
- The system shall provide tamper-detection mechanisms for security logs.
- The system shall validate cryptographic verification data before restoring plaintext content.
- The encrypted .secura file format shall support password verification, offline recovery, and orphaned-file detection.
- Sensitive database fields shall be encrypted using a database content key derived from the master password.
- Protected file contents shall remain secure during cloud backup and restore operations.
- The recovery key shall be stored only as a SHA-256 hash.
- The recovery vault shall securely store the master password in encrypted form.
- Each log entry shall contain integrity verification data.
- The system shall support tamper-evident verification by recomputing integrity hashes and comparing them with stored values.
- The system shall maintain encrypted shadow records for forensic verification and repair.
- Secure memory cleanup shall be performed after session timeout or logout.

3.1.2 Non-Functional Requirements

System Non-Functional Requirements

- The application shall provide a clear graphical user interface suitable for non-expert users.
- The application shall include an integrated Help and Manual page.
- The interface shall provide clear feedback messages for success, failure, warning, recovery, backup, and restore operations.
- Fast Lock operations should complete quickly because they rely on ACL manipulation rather than file transformation.
- Encryption and decryption operations should process files in chunks to reduce memory consumption.
- Cloud backup and restore operations should run in the background whenever possible.
- Original permissions shall be backed up before ACL modifications.
- Recovery scanning should assist in identifying incomplete operations and damaged files.
- The application shall be compatible with Windows operating systems using the NTFS file system.
- Cloud backup features shall require network access and a supported personal cloud account.
- The system should follow a modular architecture.
- The GUI, database manager, encryption module, ACL module, recovery module, log verification module, and cloud backup module should remain logically separated.
- The system shall provide practical recovery paths for password reset, missing database records, interrupted operations, orphaned encrypted files, local database loss, and device failure.

Security Non-Functional Requirements

- Encryption and decryption operations shall fail safely.
- Cloud backup shall provide an additional recovery path when local recovery is not sufficient.
- The system shall protect user data locally.
- Protected files shall not be uploaded to external servers without user authorization.
- When cloud backup is enabled, backup data shall be stored only in the user's authorized personal cloud storage.

- Backup data shall remain protected and shall not expose readable user content.
- Email OTP shall be used only for verification purposes and not for transferring user files.

3.2 System Architecture

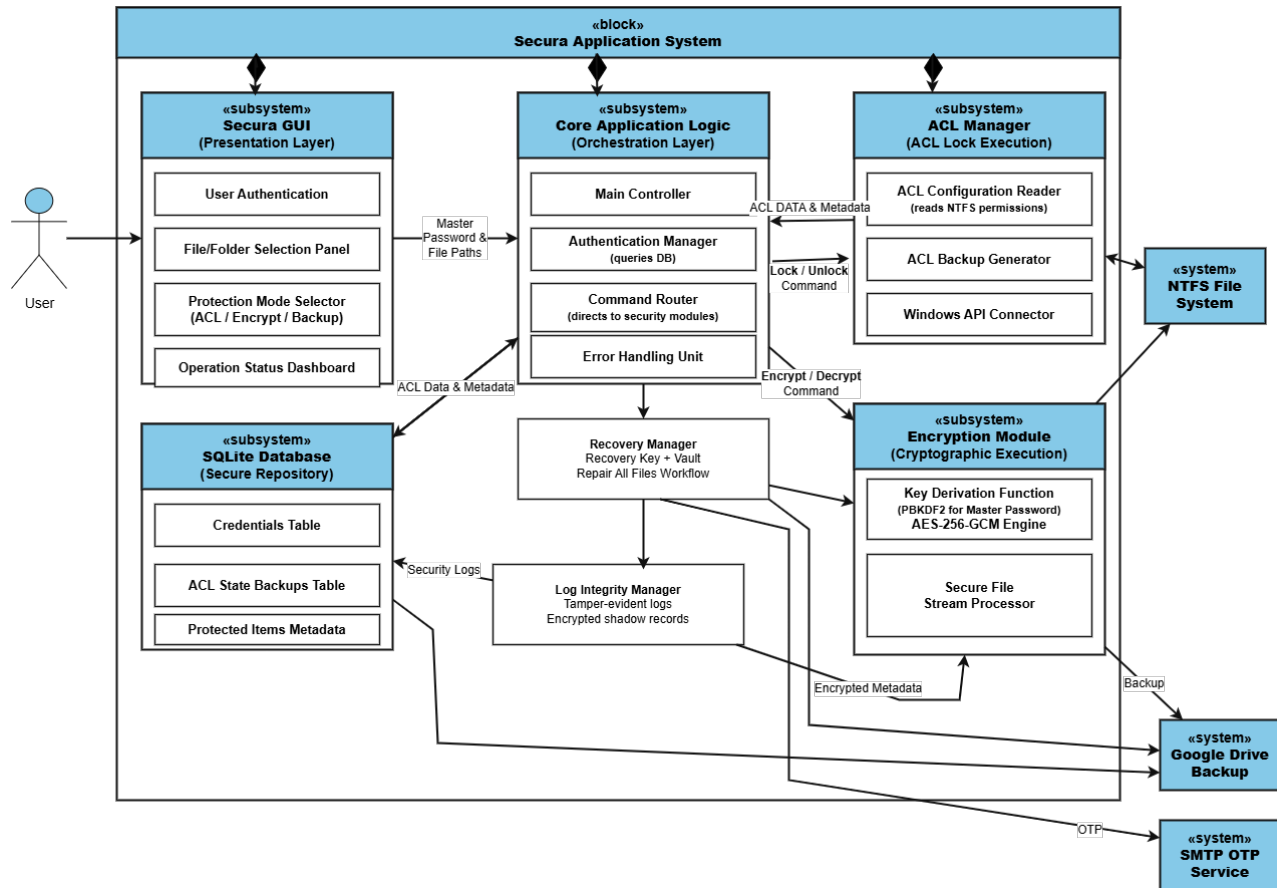


Figure 3.1: Proposed System Architecture

3.3 Use Case Diagram

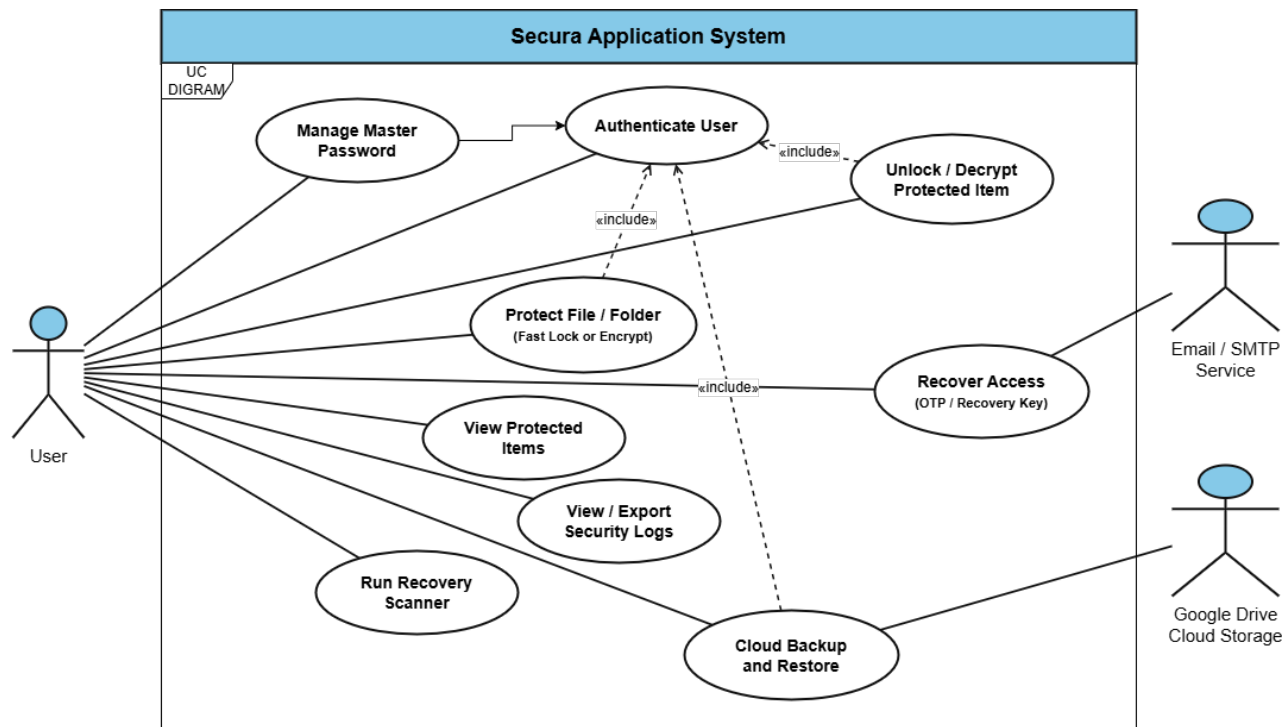


Figure 3.2: Use Case Diagram

3.4 Data Flow Diagram (DFD)

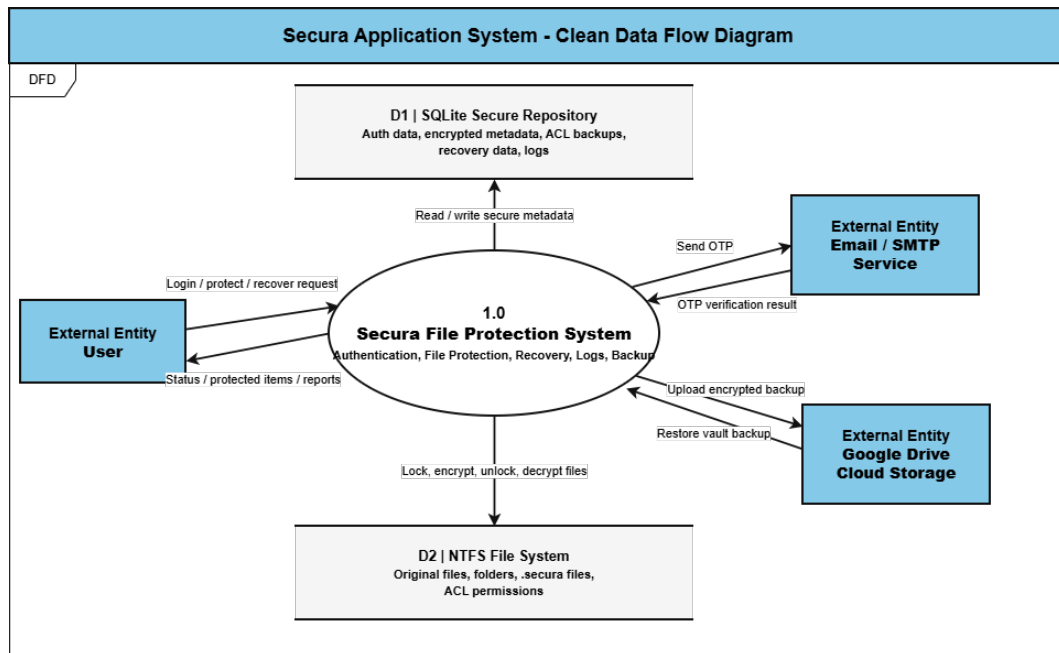


Figure 3.3: Data Flow Diagram

3.5 Security Design and Threat Model

The Secura threat model assumes that attackers may attempt to access files locally, inspect the database, modify logs, reset passwords, bypass ACLs, access encrypted files outside the application, or abuse recovery and backup features if they are not properly protected.

- **Casual Local Intrusion**

Threat: A local user attempts to open, copy, modify, or delete protected files.

Protection: Secura applies NTFS ACL DENY rules that prevent access through Windows. This is effective against casual users and shared-device risks.

- **Administrator or OS Bypass**

Threat: An attacker with high privileges removes ACL restrictions or accesses the drive outside Windows.

Protection: For highly sensitive files, AES-256-GCM encryption protects the file content. Even if ACLs are removed, the encrypted data remains unreadable without the correct password.

- **Wrong Password Attack**

Threat: An attacker attempts to decrypt a .secura file using an incorrect password.

Protection: The self-validating header and GCM authentication tags detect wrong passwords before plaintext is written.

- **Database Inspection**

Threat: An attacker opens the SQLite database to discover protected file names, paths, or logs.

Protection: Sensitive database strings are encrypted using AES-256-GCM. The attacker cannot read meaningful metadata without the master password-derived database key.

- **Log Tampering**

Threat: An attacker modifies security log records to hide activity.

Protection: Each log entry includes an integrity hash and encrypted shadow record. Tampered entries are detected during verification.

- **Password Reset Deadlock**

Threat: After password reset, old encrypted files become inaccessible because they were encrypted using the old password.

Protection: The recovery vault preserves the old master password in encrypted form and allows Repair All Files to re-encrypt old files with the new password.

- **Interrupted Operations**

Threat: Power failure or unexpected termination leaves incomplete encryption or decryption files.

Protection: The system uses safe operation handling and recovery scanning to detect temporary files, damaged files, and broken links.

- **Local Database Loss or Device Failure**

Threat: The local Secura database becomes missing, damaged, or unavailable due to storage failure, accidental deletion, or device-related problems.

Protection: The cloud backup and disaster recovery mechanism allows the system to restore critical Secura vault data from the user's authorized personal cloud storage when local recovery is not sufficient.

- **Unauthorized Cloud Backup Access**

Threat: An attacker attempts to access backup data stored outside the local device.

Protection: Secura stores backup data in a protected form and limits cloud operations to the authorized user account. Cloud backup is used to preserve recovery data without exposing readable protected file contents.

- **Cloud Authentication Token Misuse**

Threat: An attacker attempts to reuse or steal cloud authentication tokens to access backup storage.

Protection: Cloud authentication tokens are stored locally for the authenticated user and are used only to perform authorized backup and restore operations. The system restricts cloud usage to the Secura backup location and avoids unnecessary access permissions.

Chapter 4

Implementation and Methodology

4.1 Development Environment

The Secura system was implemented as a Windows desktop application using a set of technologies selected to support graphical interfaces, local storage, cryptography, operating-system integration, cloud backup, disaster recovery, and PDF export. Python 3.x [5] was used as the main programming language because it provides strong library support for desktop development, cryptography, database access, file-system operations, Windows integration, network-based services, and cloud API communication. CustomTkinter [6] was used to build the graphical user interface, as it provides modern UI components and supports a clean dark-themed interface. The final interface includes login and setup screens, a dashboard, custom popups, security logs, recovery screens, cloud backup controls, and settings panels.

SQLite [4] was used as the local database engine because it is lightweight, serverless, and suitable for desktop applications. It stores password hashes, salts, protected file metadata, recovery data, cloud backup status information, and security logs. Cryptographic operations were implemented using Python cryptography tools [7], where AES-256-GCM [3] was used for authenticated encryption, and PBKDF2-HMAC-SHA256 [15] was used for password-based key derivation. Windows integration was implemented using system-level APIs and commands, including administrator privilege checks and NTFS ACL manipulation through Windows permission tools. The `icacls` utility [12] is used to apply and remove DENY rules.

Cloud backup and disaster recovery support was implemented using Google Drive API [16] and OAuth 2.0 authorization [14]. This allows the system to connect to the user's personal cloud account, create a dedicated backup location, and upload or restore protected Secura backup data when needed. Background threading was used for cloud-related operations so that upload and restore processes do not freeze the graphical interface.

Finally, the system supports exporting logs to PDF using a PDF generation library, which helps users document operations and review security activity.

4.2 Implementation Steps

The implementation of Secura followed a structured workflow that includes application startup, authentication, interface initialization, file protection, recovery, cloud backup, logging, and session management. At startup, the system verifies Administrator privileges, initializes the local database, loads interface resources, applies the graphical theme, and prepares either the setup or login screen. The application also starts the idle-timeout mechanism and registers safe database locking operations for application shutdown.

Authentication is based on a master password. During the first launch, the user creates a password that is validated against the password policy. A random salt is generated, and the password is stored as a PBKDF2-HMAC-SHA256 hash. Password-strength guidance is provided during setup. During login, the entered password is verified using the same key derivation process. After successful authentication, the system derives the database encryption key, opens the dashboard, and keeps sensitive session data only in memory.

The graphical user interface is implemented using a state-driven design. After authentication, the system displays the main dashboard, which provides access to protected files, security logs, recovery tools, cloud backup functions, help resources, and application settings. Users can secure files or folders through a dedicated security dialog that allows selecting the target item and the required protection method.

Secura provides two protection mechanisms: Fast Lock and Deep Encryption. Fast Lock uses NTFS ACL manipulation to deny unauthorized access while preserving file contents. Before applying restrictive permissions, the original ACL state is backed up to support future restoration. Protected item metadata is then stored securely, and the operation is recorded in the security logs. For folders, optional CLSID-based obfuscation may be applied as an additional protection layer.

Deep Encryption is implemented using AES-256-GCM. The system generates a random salt, derives an encryption key using PBKDF2-HMAC-SHA256, creates the required nonce values, and stores the encrypted output in a custom self-validating .secura file format. The file header contains cryptographic validation data that allows password verification before decrypting the actual content. File processing is performed in chunks to reduce memory consumption. After successful encryption, the original file is removed and the encrypted file is additionally protected using ACL rules.

The system also supports a fast encryption mode in which only the first 4096 bytes of a file are encrypted. This approach prevents normal file recognition while reducing processing time for large files. The encryption mode is automatically detected during decryption.

To protect stored metadata, sensitive database fields such as file names, file paths, protection status, and log details are encrypted using AES-256-GCM with a database content key derived from the master password. This prevents meaningful information from being exposed if the SQLite database is accessed directly.

Cloud backup and disaster recovery are integrated into the recovery framework. Users can authorize access to their cloud account through OAuth 2.0, after which the system creates or uses a dedicated backup location. Backup operations upload encrypted vault data, recovery information, and logs, while restore operations download and safely replace damaged or missing local records. Cloud tasks execute in background threads to maintain interface responsiveness.

Recovery functionality includes recovery keys, QR-code export, recovery vaults, email

OTP verification, and password reset support. During setup, the system generates a 24-character recovery key and stores only its SHA-256 hash. The recovery vault securely stores the master password in encrypted form and enables password recovery when repairing old encrypted files. The recovery key can also be exported as a QR Code to simplify secure storage and transfer. Email OTP verification is implemented using six-digit codes transmitted through SMTP over STARTTLS.

The Repair All Files mechanism addresses master-password changes by decrypting existing .secura files with the old password and re-encrypting them using the new password. The process also updates encrypted database content, recovery information, and recovery credentials.

Finally, Secura implements security logging, recovery scanning, cloud auditing, and automatic logout. Security logs record important operations and include integrity verification data to detect tampering. The recovery scanner analyzes both the database and file system to identify missing, orphaned, damaged, or incomplete files. The automatic logout mechanism monitors user activity and terminates inactive sessions by clearing sensitive data from memory and requiring re-authentication.

4.3 Algorithms and Models Used

This section presents the main algorithms used in the implementation of the Secura system. The algorithms are expressed using high-level pseudocode to describe the logical flow of both the ACL manipulation and the cryptographic data transformations.

The following algorithm describes the process of creating and securely storing the master password during the initial setup phase.

Algorithm 1 Master Password Creation

Input: User password P

Output: Stored password hash and salt

1. Receive password from user
 2. Check password policy
 3. Generate random 16-byte salt S
 4. Compute password hash $H = \text{PBKDF2-HMAC-SHA256}(P, S)$
 5. Store salt S and hash H in the settings table
 6. Generate recovery key
 7. Store recovery key hash
 8. Create recovery vault
 9. Open application dashboard
-

The following algorithm describes the user authentication and login verification process.

Algorithm 2 User Login

Input: Entered password P'

Output: Authenticated session or failure

1. Retrieve stored salt S and hash H from database
 2. Compute candidate hash $H' = \text{PBKDF2-HMAC-SHA256}(P', S)$
 3. Compare candidate hash H' with stored hash H using constant-time comparison
 4. If $H' = H$, derive database content key
 5. Store master password in memory for the session
 6. Open dashboard
 7. If $H' \neq H$, reject login
-

The following algorithm describes the Fast Lock mechanism used to quickly protect files and folders using NTFS ACL permissions.

Algorithm 3 Fast Lock Operation

Input: Target file or folder path T

Output: ACL-protected item

1. Authenticate user
 2. Verify Administrator privileges
 3. Check that the target path T exists
 4. Read current ACL permissions of T
 5. Store ACL backup in database
 6. Apply DENY permissions to the target item
 7. Store encrypted metadata in database
 8. Add security log entry
 9. Display success message
-

The following algorithm describes the process of restoring original permissions and unlocking protected files or folders.

Algorithm 4 Unlock Operation

Input: Protected item path T

Output: Restored item permissions

1. Authenticate user
 2. Retrieve ACL backup from database
 3. Remove restrictive DENY rules
 4. Restore original permissions
 5. Update item status in database
 6. Add security log entry
 7. Display result to user
-

The following algorithm describes the AES-256-GCM encryption process.

Algorithm 5 AES-256-GCM Encryption

Input: Plaintext file path T_{plain} , master password P

Output: .secura encrypted file T_{enc}

1. Authenticate user
 2. Generate random salt S
 3. Derive AES-256 key K using PBKDF2-HMAC-SHA256
 4. Generate nonce values
 5. Encrypt validation word and generate validation tag
 6. Write .secura header
 7. Read plaintext file in chunks
 8. Encrypt each chunk using AES-256-GCM
 9. Write ciphertext to output file
 10. Append authentication tag
 11. Verify successful completion
 12. Delete original plaintext file
 13. Apply ACL protection to encrypted file
 14. Store metadata and log operation
-

The following algorithm describes the AES-256-GCM decryption process.

Algorithm 6 AES-256-GCM Decryption

Input: .secura file T_{enc} , master password P

Output: Restored plaintext file T_{plain}

1. Authenticate user
 2. Read .secura header
 3. Validate magic value
 4. Derive AES key using password and stored salt
 5. Decrypt validation word
 6. Verify validation authentication tag
 7. If validation fails, stop operation
 8. Decrypt file content in chunks
 9. Verify data authentication tag
 10. Write restored plaintext file
 11. Update database and logs
-

The following algorithm describes the recovery key verification process used to validate user identity before performing recovery operations.

Algorithm 7 Recovery Key Verification

Input: Recovery key R

Output: Verification result

1. Receive recovery key from user
 2. Normalize key format
 3. Compute SHA-256 hash of the entered key
 4. Compare with stored recovery hash
 5. If equal, allow recovery workflow
 6. If not equal, reject recovery attempt
-

The following algorithm describes the process of generating and exporting the recovery key as a QR Code for secure storage and transfer.

Algorithm 8 Recovery Key QR Export

Input: Recovery key R

Output: QR Code image containing the recovery key

1. Retrieve recovery key R
 2. Format the recovery key as readable text
 3. Append the label “Secura Recovery Key:”
 4. Generate QR Code data
 5. Create QR Code image
 6. Render QR Code in the graphical interface
 7. Display scanning instructions to the user
 8. Allow secure transfer of the recovery key to a mobile device
-

The following algorithm describes the process of repairing encrypted files after a master password change.

Algorithm 9 Repair All Files

Input: Old password or recovery key, new master password

Output: Files re-encrypted with new password

1. Verify recovery key or old password
 2. Retrieve old password from recovery vault if needed
 3. Re-encrypt database strings using the new database key
 4. For each encrypted file:
 - (a) Remove ACL DENY rules temporarily
 - (b) Decrypt file using old password
 - (c) Re-encrypt file using new password
 - (d) Reapply ACL DENY rules
 - (e) Update database record
 5. Generate new recovery key
 6. Create new recovery vault
 7. Add log entry
 8. Display repair summary
-

The following algorithm describes the log integrity verification process.

Algorithm 10 Log Integrity Check

Input: Stored log records

Output: Valid or tampered status

1. Retrieve log records from database
 2. For each log entry, recompute integrity hash
 3. Compare recomputed hash with stored hash
 4. If equal, mark log as valid
 5. If not equal, mark log as tampered
 6. Display result in logs interface
-

The following algorithm describes the recovery scanning process.

Algorithm 11 Recovery Scan

Input: Database records and file system paths

Output: Recovery report

1. Retrieve protected file records
 2. Check whether each referenced file exists
 3. Scan disk for .secura files
 4. Identify orphaned encrypted files
 5. Check temporary files from interrupted operations
 6. Validate minimum .secura file structure
 7. Display issues and recommended actions
-

The following algorithm describes the cloud authentication process.

Algorithm 12 Cloud Authentication

Input: User cloud authorization request

Output: Authorized cloud session or failure

1. Receive cloud connection request from user
 2. Check whether a valid cloud authentication token already exists
 3. If the token exists and is valid, reuse it for backup and restore operations
 4. If the token is missing or expired, start the OAuth authorization flow
 5. Open the authorization page for the user
 6. Receive the authorization response from Google Drive
 7. Store the authentication token locally for future cloud operations
 8. Create or locate the Secura cloud backup folder
 9. Display cloud connection status to the user
-

The following algorithm describes the cloud backup and auto-sync process.

Algorithm 13 Cloud Backup and Auto-Sync

Input: Local Secura database and recovery-related data

Output: Updated cloud backup copy

1. Authenticate user
 2. Verify that cloud authentication is available
 3. Check or create the dedicated Secura cloud backup folder
 4. Prepare the local backup data
 5. Ensure that sensitive backup data remains protected before upload
 6. Upload the backup data to the user's personal cloud storage
 7. Replace the previous cloud backup copy if needed
 8. Record cloud backup operation in the security logs
 9. Display backup success or failure message to the user
-

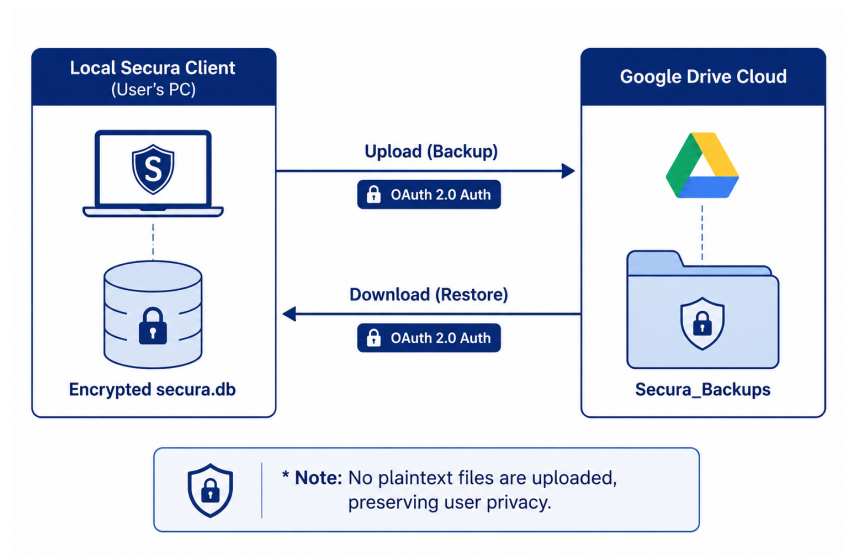


Figure 4.1: Cloud Backup and Auto-Sync

The following algorithm describes the cloud disaster recovery process.

Algorithm 14 Cloud Disaster Recovery

Input: Cloud backup copy

Output: Restored local Secura vault data

1. Start disaster recovery workflow
 2. Verify cloud authentication
 3. Locate the Secura backup folder in cloud storage
 4. Download the latest available backup copy
 5. Store the downloaded backup temporarily
 6. Close active database connections if required
 7. Replace the missing or damaged local database with the restored copy
 8. Reload the database state
 9. Record cloud restore operation in the security logs
 10. Display restore result to the user
-

The following algorithm describes the cloud backup error-handling process.

Algorithm 15 Cloud Backup Error Handling

Input: Cloud backup or restore operation status

Output: Safe failure handling and user feedback

1. Detect network, authorization, upload, or download failure
 2. Stop the current cloud operation safely
 3. Keep the existing local database unchanged
 4. Preserve the last valid backup when possible
 5. Record the failure in the security logs
 6. Display a clear error or warning message to the user
 7. Allow the user to retry the cloud backup or restore operation later
-

4.4 Security Mechanisms

Secura integrates multiple security mechanisms to protect user data and maintain system integrity. AES-256-GCM authenticated encryption provides confidentiality and integrity for deeply protected files by encrypting file contents and generating authentication tags. Any modification to encrypted data or use of an incorrect password causes authentication failure, preventing invalid decryption. PBKDF2-HMAC-SHA256 is used to derive cryptographic keys from passwords and recovery keys using random salts and high iteration counts to resist brute-force and rainbow-table attacks [17].

For fast local protection, the system uses NTFS ACL DENY rules to block unauthorized access without modifying file contents. The custom .secura file format includes password-validation data, allowing password verification before decryption and supporting recovery even when database records are unavailable. Sensitive metadata, including file names, file paths, encrypted paths, and log details, is encrypted before storage in SQLite.

To support recovery, Secura uses a recovery vault that stores the master password encrypted with a key derived from the recovery key. The recovery key can also be exported as a locally generated QR Code, enabling secure transfer and storage without network communication. Security logs include integrity hashes to detect tampering, while encrypted shadow records preserve protected copies of original log data for forensic verification.

For cloud backup and disaster recovery, the system uses secure cloud authorization and uploads protected backup data to a dedicated Secura backup location. Authentication tokens are stored locally and used only for backup and restore operations, while all cloud activities are recorded in the security logs for auditing and forensic review.

Finally, Secura implements automatic logout, secure file-processing procedures, and recovery scanning. Inactive sessions are terminated automatically, sensitive memory contents are cleared, and re-authentication is required. Encryption, decryption, and cloud recovery operations use controlled validation and recovery procedures to reduce corruption risks and preserve data integrity.

4.5 User Interface and System Screens

The Secura system includes a user-friendly graphical interface that allows users to manage file protection, recovery operations, security logs, cloud synchronization, and system settings. Detailed screenshots and descriptions of the implemented interfaces are provided in **Appendix B**.

Chapter 5

Testing, Evaluation, and Results

5.1 Testing Strategy

The Secura system was tested to make sure that the main features work correctly and safely. The testing focused on authentication, file locking, file encryption, file unlocking, decryption, recovery, logs, and cloud backup.

Functional testing was used to check that each feature performs its task as expected. Security testing was used to make sure that wrong passwords are rejected, protected files cannot be accessed normally, and encrypted files cannot be opened without the correct password. Recovery testing was also performed to check password reset, recovery key verification, and file repair after password changes.

5.2 Experimental Setup

The experimental evaluation of the Secura system was conducted in a controlled local environment to ensure reliable, consistent, and reproducible performance measurements. All tests were performed on a desktop computer equipped with a 13th Generation Intel® Core™ i7-13700KF processor operating at a base frequency of 3.40 GHz, providing sufficient computational power for AES-256-GCM encryption, PBKDF2 key derivation, and other security-related operations. The system was equipped with 32 GB of DDR5 RAM, allowing efficient memory management and stable execution of file protection and recovery processes.

For storage, a high-speed NVMe M.2 SSD was used to minimize disk input/output latency and reduce storage-related bottlenecks during file encryption, decryption, backup, and recovery operations. The software environment consisted of Microsoft Windows 11 (64-bit), which provides native support for the NTFS file system and Access Control List (ACL) mechanisms required by the Fast Lock protection mode. The Secura application was implemented and executed using Python 3, while SQLite was used as the local database engine for storing authentication data, protected file metadata, recovery information, and security logs.

The testing process included multiple file sizes and protection scenarios in order to evaluate both ACL-based Fast Lock operations and AES-256-GCM Deep Encryption operations. Performance measurements were collected for execution time, cryptographic throughput, memory utilization, and storage overhead under normal operating conditions.

5.3 Performance Metrics

The performance evaluation of the Secura system was conducted using a set of metrics designed to measure efficiency, resource consumption, and storage impact. Since the system combines two different protection approaches, namely Fast Lock based on NTFS Access Control Lists (ACLs) and Deep Encryption based on AES-256-GCM, the evaluation focused on practical performance characteristics rather than traditional accuracy measurements. The first metric was Execution Time (Latency), which measures the total time required to complete a protection or recovery operation. This metric was used to highlight the performance difference between the ACL-based Fast Lock mechanism, which operates almost instantly, and the cryptographic encryption process, which requires additional processing time depending on file size.

The second metric was Cryptographic Throughput, measured in Megabytes per second (MB/s). This metric represents the amount of data that can be encrypted or decrypted within a specific period of time and provides an indication of the efficiency of the AES-256-GCM cryptographic engine when handling files of various sizes. Higher throughput values indicate better cryptographic processing performance and improved scalability for large files. Another important metric was Resource Utilization, specifically Peak Memory Usage. This metric measures the maximum amount of RAM consumed during encryption and decryption operations. It was included to evaluate the effectiveness of the chunk-based file processing mechanism implemented in Secura. By processing files in smaller chunks rather than loading entire files into memory, the system is expected to maintain low and stable memory consumption even when operating on large files.

Finally, Storage Overhead was evaluated to determine the additional space introduced by the custom .secura encrypted file format. This metric measures the difference between the original file size and the size of the encrypted output. The purpose of this evaluation is to verify that the security-related metadata embedded within the .secura file, including the magic value, cryptographic salt, nonce values, validation ciphertext, and authentication tags, introduces only a minimal and predictable increase in storage requirements while preserving the security and integrity of protected files.

5.4 Results and Analysis

This section presents the experimental results obtained from evaluating the Secura file protection system under different workloads. The evaluation focused on comparing the performance of the two protection mechanisms implemented in the system, namely NTFS ACL-based Fast Lock and AES-256-GCM Deep Encryption. Tests were performed using files of different sizes (10 MB, 100 MB, and 500 MB) to assess scalability, processing efficiency, memory utilization, and storage impact.

5.4.1 Execution Time Comparison (Fast Lock vs. Deep Encryption)

The first experiment evaluated the execution time required to protect files using both Fast Lock and Deep Encryption. Table 5.1 presents the measured execution times for files of different sizes.

Table 5.1: Execution Time Comparison

File Size	Fast Lock Time (s)	Deep Encryption Time (s)
10 MB	0.007	0.034
100 MB	0.006	0.091
500 MB	0.006	0.481

Analysis:

The results demonstrate that the Fast Lock mechanism maintains an almost constant execution time regardless of file size. Since ACL-based protection only modifies file permissions and metadata without processing file contents, the operation completes within approximately 0.006 seconds for all tested files. This confirms that the performance of Fast Lock is largely independent of file size and is suitable for instant file protection.

In contrast, the execution time of Deep Encryption increases as file size grows because AES-256-GCM must process the entire file content. The encryption time increased from 0.034 seconds for a 10 MB file to 0.481 seconds for a 500 MB file. Despite this increase, the encryption process remained highly efficient, completing the protection of a 500 MB file in less than half a second. These results validate the system's dual-layer security design, where Fast Lock provides immediate protection for everyday use, while Deep Encryption delivers strong confidentiality with acceptable performance overhead.

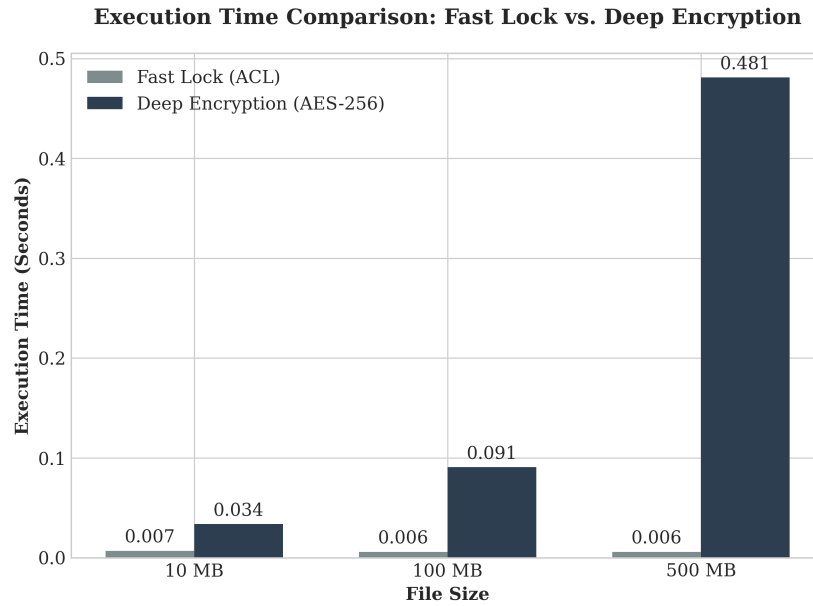


Figure 4.5.1: Results and Analysis

5.4.2 Cryptographic Engine Performance

The second experiment focused on evaluating the efficiency of the cryptographic engine through throughput, memory consumption, and storage overhead measurements.

Table 5.2: Cryptographic Performance Metrics

File Size	Throughput (MB/s)	Peak RAM Usage (MB)	Storage Overhead (Bytes)
10 MB	293.38	0.78	86
100 MB	1102.45	0.14	86
500 MB	1040.18	0.14	86

Analysis:

The throughput results indicate that the AES-256-GCM implementation achieves high processing efficiency across different file sizes. Throughput exceeded 1 GB/s for larger files, reaching 1102.45 MB/s for the 100 MB test file and remaining above 1040 MB/s for the 500 MB file. The slight variation between measurements can be attributed to operating system scheduling and storage access fluctuations. Overall, the results demonstrate that the cryptographic module scales effectively as file size increases.

The memory utilization results further confirm the effectiveness of the chunk-based processing mechanism implemented in Secura. Peak RAM consumption remained extremely low throughout the experiments and did not increase significantly with larger files. This behavior indicates that the application processes data incrementally rather than loading entire files into memory, allowing efficient handling of large files while maintaining system stability.

Finally, the storage overhead remained constant at 86 bytes for all tested file sizes. This overhead corresponds to the cryptographic metadata embedded within the custom .secura file format, including the file header, salt values, nonce values, validation ciphertext, and authentication tags. The results confirm that the security features introduce only a minimal and predictable storage cost, regardless of the size of the protected file.

Overall, the experimental results demonstrate that Secura successfully achieves its design objectives by combining near-instant ACL-based protection with highly efficient AES-256-GCM encryption. The system maintains low resource consumption, high cryptographic throughput, and minimal storage overhead, making it suitable for protecting both small and large files in real-world environments.

5.5 Security Evaluation

The system provides confidentiality by using AES-256-GCM encryption for sensitive files and ACL rules for fast local protection. It also provides authentication through the master password, which is stored as a secure hash instead of plaintext. Integrity is supported by AES-GCM authentication tags and log integrity checking. If an encrypted file or log record is modified, the system can detect the change. The system also improves recoverability through the recovery key, email OTP verification, Repair All Files feature, and cloud backup. These features help reduce the risk of losing access to protected files. Overall, the testing showed that Secura meets the main security and usability goals of the project.

Chapter 6

Conclusion and Future Work

6.1 Summary of Work

This project presented Secura, a Windows desktop application for protecting local files and folders. The system provides two protection methods: fast locking using NTFS ACL rules and deep encryption using AES-256-GCM.

The application also includes a master password system, encrypted database metadata, security logs, recovery key support, email OTP verification, file repair, and cloud backup. These features make the system more secure and more practical for real-world use.

6.2 Conclusions

The Secura system successfully achieved its primary objective of providing a secure, user-friendly, and efficient file protection solution for Windows environments. By combining NTFS ACL-based Fast Lock with AES-256-GCM Deep Encryption, the system offers two complementary protection mechanisms that address different security requirements while maintaining ease of use.

The implementation demonstrated that ACL-based protection provides near-instant access restriction with minimal resource consumption, while AES-256-GCM encryption delivers strong confidentiality and integrity protection for highly sensitive data. Performance evaluation showed that the system can protect large files efficiently while maintaining low memory usage and minimal storage overhead.

The project also highlighted the importance of recovery and maintainability in security

applications. Through the integration of recovery keys, OTP verification, repair mechanisms, recovery scanning, and cloud backup, Secura reduces the risk of permanent data loss and improves system resilience in real-world scenarios.

Overall, Secura provides a balanced combination of security, performance, usability, and recoverability. The developed architecture establishes a solid foundation for future enhancements and demonstrates the practicality of integrating access control, cryptographic protection, and recovery services within a single unified platform.

6.3 Future Work

In the future, the system can be improved by adding multi-user support, so each user can have separate protected files and settings. Another possible improvement is adding per-file passwords instead of using only one master password.

The cloud backup and disaster recovery module can be further enhanced by supporting additional cloud storage providers such as Microsoft OneDrive and Dropbox alongside Google Drive. This enhancement would provide users with greater flexibility in selecting their preferred backup platform while increasing backup redundancy and system availability. Furthermore, future versions of Secura may implement multi-cloud backup support, allowing recovery data to be securely replicated across multiple cloud services to improve resilience against service outages and account-related failures.

Finally, the system can be extended to support other operating systems such as Linux and macOS, while keeping encryption as the main protection method.

Bibliography

- [1] Ross Brewer. Ransomware attacks: detection, prevention and cure. *Network Security*, 2016(9):5–9, 2016.
- [2] Microsoft Corporation. Access control lists - win32 apps, 2023. [Accessed: 20-Oct-2025].
- [3] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/-counter mode (gcm) and gmac. Technical Report SP 800-38D, National Institute of Standards and Technology (NIST), 2007.
- [4] SQLite Consortium. Sqlite official documentation, 2024. [Accessed: 20-Oct-2025].
- [5] Python Software Foundation. Python 3.12.x documentation, 2024. [Accessed: 20-Oct-2025].
- [6] Tom Schimansky. Customtkinter: A modern and customizable python ui-library based on tkinter, 2024. [Accessed: 20-Oct-2025].
- [7] Python Cryptographic Authority. Cryptography library documentation, 2024. [Accessed: 20-Oct-2025].
- [8] William Stallings. *Cryptography and Network Security: Principles and Practice*. Pearson, 8th edition, 2020.
- [9] Abraham Silberschatz, Peter B Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley & Sons, 10th edition, 2018.
- [10] David A McGrew and John Viega. The security and performance of the galois/-counter mode (gcm) of operation. In *Progress in Cryptology–INDOCRYPT 2004*, pages 343–355. Springer, 2004.
- [11] IDRIX. Veracrypt documentation, 2024. [Accessed: 20-Oct-2025].
- [12] Microsoft Corporation. icacils - windows it pro center, 2023. [Accessed: 20-Oct-2025].
- [13] Kui Ren, Cong Wang, and Qian Wang. Data security and privacy in cloud computing. *IEEE wireless communications*, 19(1):69–74, 2012.
- [14] D. Hardt. The oauth 2.0 authorization framework. RFC 6749, Internet Engineering Task Force (IETF), 2012.

- [15] K. Moriarty and A. Kaliski. Pkcs #5: Password-based cryptography specification version 2.1. RFC 8018, Internet Engineering Task Force (IETF), 2017.
- [16] Google Developers. Google drive api documentation, 2024. [Accessed: 20-Oct-2025].
- [17] Matt Weir, Sudhir Aggarwal, Breno De Medeiros, and Bill Glodek. Password cracking using probabilistic context-free grammars. In *2009 IEEE Symposium on Security and Privacy*, pages 391–405. IEEE, 2009.

Appendix A

Source Code

This appendix presents selected source code segments from the Secura system. The included implementations represent the core security, protection, recovery, and cloud backup mechanisms developed during the project.

A.1 Master Password Creation and Authentication

Explanation

This component handles the secure creation and verification of the user's master credentials using standard key-stretching techniques. It generates a cryptographically secure random salt and applies PBKDF2-HMAC-SHA256 to strengthen password security against brute-force and dictionary attacks. Authentication is performed using constant-time comparison to reduce the risk of timing attacks.

Source Code

```
def _hash_password(password: str, salt_hex: Optional[str] = None) -> Tuple[str, str]:
    if salt_hex:
        salt = bytes.fromhex(salt_hex)
    else:
        salt = os.urandom(16)

    salt_hex_out = salt.hex()

    dk = hashlib.pbkdf2_hmac(
        "sha256",
        password.encode("utf-8"),
        salt,
        200_000,
        dklen=32,
    )

    return dk.hex(), salt_hex_out
```

```

def verify_master_password(conn: sqlite3.Connection, password: str) -> bool:
    try:
        cur = conn.cursor()

        cur.execute(
            "SELECT master_password_hash, master_salt FROM settings
            WHERE id = 1 LIMIT 1"
        )

        row = cur.fetchone()

        if not row:
            return False

        stored_hash_hex = row["master_password_hash"]
        salt_hex = row["master_salt"]

        candidate_hash_hex, _ = _hash_password(password, salt_hex)

        return hmac.compare_digest(
            stored_hash_hex,
            candidate_hash_hex
        )

    except Exception:
        return False

```

A.2 AES-256-GCM File Encryption Algorithm

Explanation

This module implements authenticated file encryption using AES-256-GCM. A 256-bit key is derived from the master password, and the file is encrypted in chunks to maintain low memory consumption while supporting large files.

Source Code

```
def _derive_key(password: str, salt: bytes) -> bytes:

    kdf = PBKDF2HMAC(
        algorithm=hashes.SHA256(),
        length=32,
        salt=salt,
        iterations=100_000,
        backend=default_backend(),
    )

    return kdf.derive(password.encode("utf-8"))


def encrypt_file(source_path: str,
                 password: str,
                 fast_mode: bool = False) -> str:

    salt = secrets.token_bytes(SALT_LEN)
    header_nonce = secrets.token_bytes(HEADER_NONCE_LEN)
    data_nonce = secrets.token_bytes(NONCE_LEN)

    key = _derive_key(password, salt)

    encrypted_path = source_path + ".secura"

    with open(source_path, "rb") as src, \
        open(encrypted_path, "wb") as dst:

        dst.write(MAGIC)
        dst.write(salt)
        dst.write(header_nonce)

        dat_cipher = Cipher(
            algorithms.AES(key),
            modes.GCM(data_nonce),
            backend=default_backend()
        )
```

```
dat_enc = dat_cipher.encryptor()

while True:
    chunk = src.read(CHUNK_SIZE)

    if not chunk:
        break

    dst.write(dat_enc.update(chunk))

dat_enc.finalize()

dst.write(dat_enc.tag)

os.remove(source_path)

return encrypted_path
```

A.3 AES-256-GCM File Decryption Algorithm

Explanation

This module restores encrypted files by validating the file header, verifying authentication tags, and performing chunk-based decryption. Integrity verification prevents unauthorized modification of encrypted data.

Source Code

```
def _decrypt_file_impl(encrypted_path: str,
                      password: str) -> str:

    restored_path = encrypted_path.replace(".secura", "")

    with open(encrypted_path, "rb") as src:

        salt = src.read(SALT_LEN)

        key = _derive_key(password, salt)

        dat_cipher = Cipher(
            algorithms.AES(key),
            modes.GCM(data_nonce, data_tag),
            backend=default_backend()
        )

        dat_dec = dat_cipher.decryptor()

        with open(restored_path, "wb") as dst:

            while True:
                chunk = src.read(CHUNK_SIZE)

                if not chunk:
                    break

                dst.write(dat_dec.update(chunk))

            dat_dec.finalize()

    os.remove(encrypted_path)

    return restored_path
```


A.4 NTFS ACL Fast Lock and Unlock Implementation

Explanation

This subsystem provides operating-system-level protection using NTFS Access Control Lists (ACLs). It restricts file access by applying and removing permission rules through the Windows `icacls` utility.

Source Code

```
def _apply_acl_lock(path: str) -> bool:

    abs_path = os.path.abspath(path)

    try:

        subprocess.run(
            ["icacls",
             abs_path,
             "/deny",
             "*S-1-1-0:(R,W)"],
            check=True,
            capture_output=True,
            text=True,
        )

        return True

    except subprocess.CalledProcessError:
        return False

def _apply_acl_unlock(path: str) -> bool:

    abs_path = os.path.abspath(path)

    try:

        subprocess.run(
            ["icacls",
             abs_path,
             "/remove:d",
             "*S-1-1-0"],
            check=True,
            capture_output=True,
            text=True,
        )

        return True
```

```
except subprocess.CalledProcessError:  
    return False
```

A.5 Recovery Key and Recovery Vault Mechanism

Explanation

This mechanism supports secure password recovery by generating a recovery key and using it to encrypt the master password inside a protected recovery vault.

Source Code

```
def generate_recovery_key_logic() -> str:

    groups = [
        ''.join(
            random.choices(
                string.ascii_uppercase +
                string.digits,
                k=4
            )
        )
        for _ in range(6)
    ]

    return "-".join(groups)


def encrypt_master_key_with_recovery_key(
    master_password: str,
    recovery_key: str) -> str:

    key = _derive_vault_key(recovery_key)

    nonce = secrets.token_bytes(12)

    cipher = Cipher(
        algorithms.AES(key),
        modes.GCM(nonce),
        backend=default_backend()
    )

    enc = cipher.encryptor()

    ct = enc.update(
        master_password.encode("utf-8")
    ) + enc.finalize()

    return (nonce + enc.tag + ct).hex()
```

```

def decrypt_master_key_with_recovery_key(
    vault_hex: str,
    recovery_key: str) -> str:

    data = bytes.fromhex(vault_hex)

    nonce = data[:12]
    tag = data[12:28]
    ct = data[28:]

    key = _derive_vault_key(recovery_key)

    cipher = Cipher(
        algorithms.AES(key),
        modes.GCM(nonce, tag),
        backend=default_backend()
    )

    dec = cipher.decryptor()

    return (
        dec.update(ct) +
        dec.finalize()
    ).decode("utf-8")

```

A.6 Google Drive Backup and Disaster Recovery Module

Explanation

This module provides secure cloud backup and disaster recovery functionality using Google Drive. It supports OAuth 2.0 authentication, remote backup synchronization, and restoration of critical Secura vault data.

Source Code

```
class GoogleDriveManager:

    def connect(self):

        flow = InstalledAppFlow.from_client_secrets_file(
            str(CREDENTIALS_PATH),
            SCOPES
        )

        creds = flow.run_local_server(
            port=0,
            open_browser=True
        )

        TOKEN_PATH.write_text(
            creds.to_json(),
            encoding="utf-8"
        )

    def upload_database(self,
                        db_path: str) -> str:

        service = self._get_service()

        media = MediaFileUpload(
            db_path,
            mimetype="application/x-sqlite3"
        )

        created = service.files().create(
            body={
                "name": Path(db_path).name
            },
            media_body=media,
            fields="id"
        ).execute()
```

```

        return created["id"]

def download_database(self,
                      dest_path: str) -> str:

    service = self._get_service()

    request = service.files().get_media(
        fileId=file_id
    )

    with open(dest_path, "wb") as fh:

        downloader = MediaIoBaseDownload(
            fh,
            request
        )

        done = False

        while not done:
            _, done = downloader.next_chunk()

    return dest_path

```

Appendix B

User Guide

This appendix provides a practical guide for operating the Secura file protection system. It explains the main features and workflows available to end users, including authentication, file protection, cloud backup, recovery procedures, and security monitoring.

B.1 First-Time Setup

When launching Secura for the first time, the user must initialize the secure vault by creating a master password and configuring recovery options.

Set Your Master Password

Your master password encrypts and protects all your data.
Choose a strong password you'll remember — it cannot be recovered.

Enter Master Password

Enter a strong password

- ✗ Minimum 12 characters
- ✗ At least one uppercase letter (A-Z)
- ✗ At least one lowercase letter (a-z)
- ✗ At least one number (0-9)
- ✗ At least one symbol / special character

Confirm Master Password

Re-enter your password

Email Address

you@example.com **Send OTP**

Initialize System

[Disaster Recovery](#)

[Help & Manual](#)

Secura • Secure Password Management

Figure 4.5.1: First-Time Setup

Figure 4.5.1 illustrates the initial setup process used to create the secure vault and configure recovery information.

B.2 Recovery Key and QR Code Screen

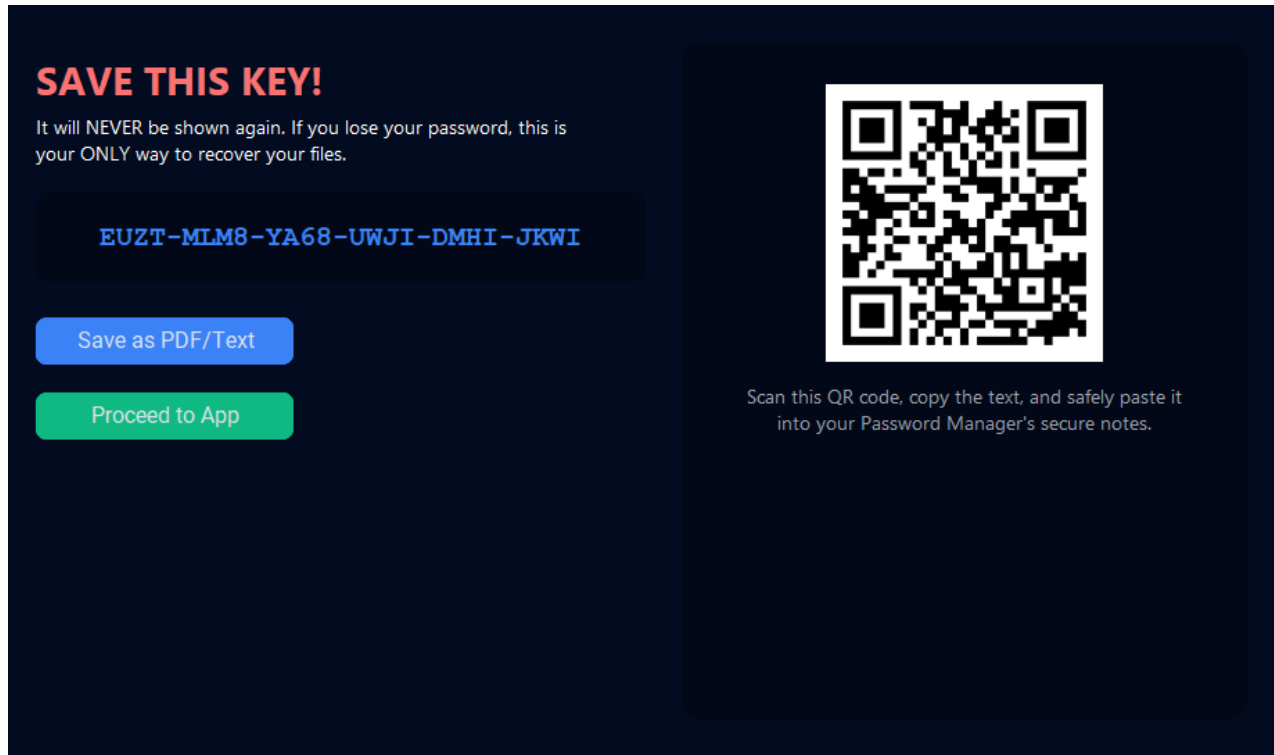


Figure 4.5.2: Login Interface

Figure 4.5.2 shows the Recovery Key interface. The screen displays the generated recovery key and its QR code representation for secure storage and future account recovery.

B.3 Managing Protected Files

The dashboard provides centralized management of all protected files and folders.

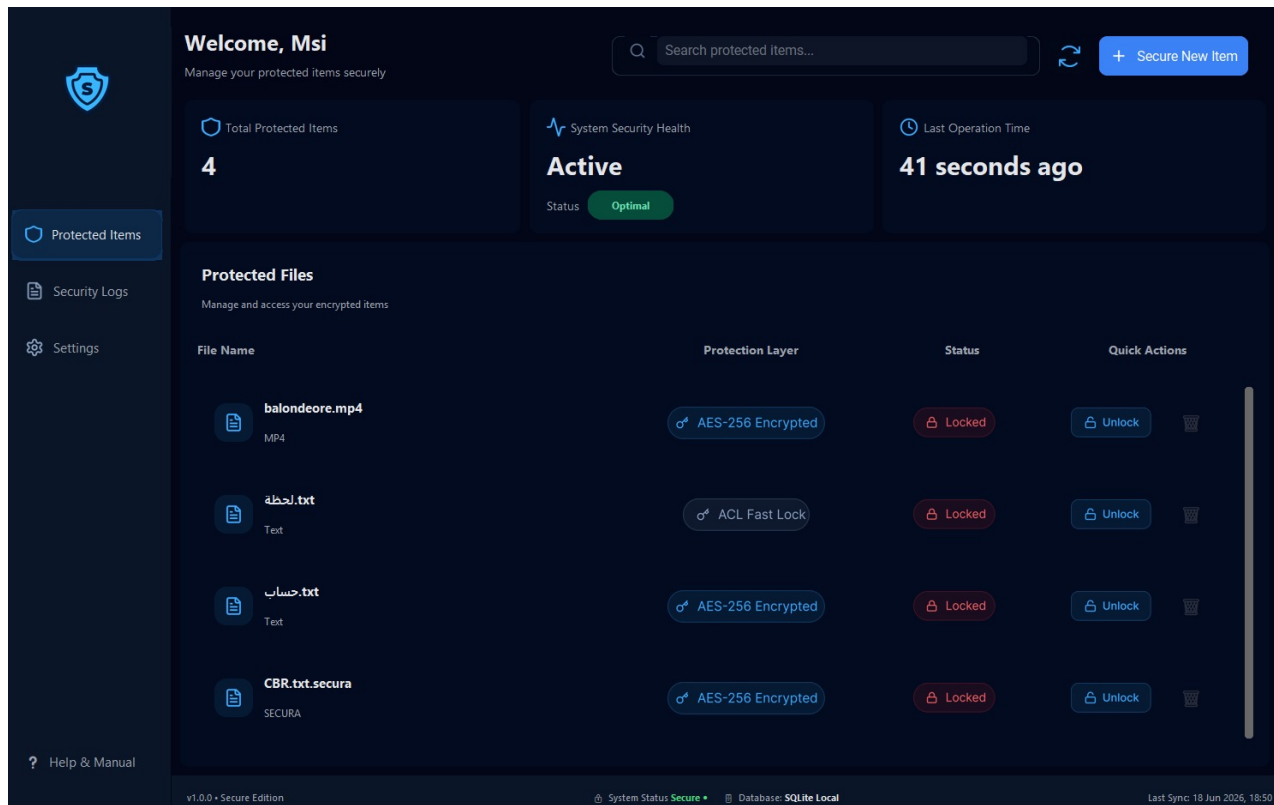


Figure 4.5.3: Main Dashboard

Figure 4.5.3 presents the main dashboard displaying protected files, security status information, and available management actions.

B.4 File Protection Operations

Secura provides two protection mechanisms:

- Fast Lock (ACL-Based Protection)
- Deep Encryption (AES-256-GCM)

Users can select the desired protection method when securing a file or folder.

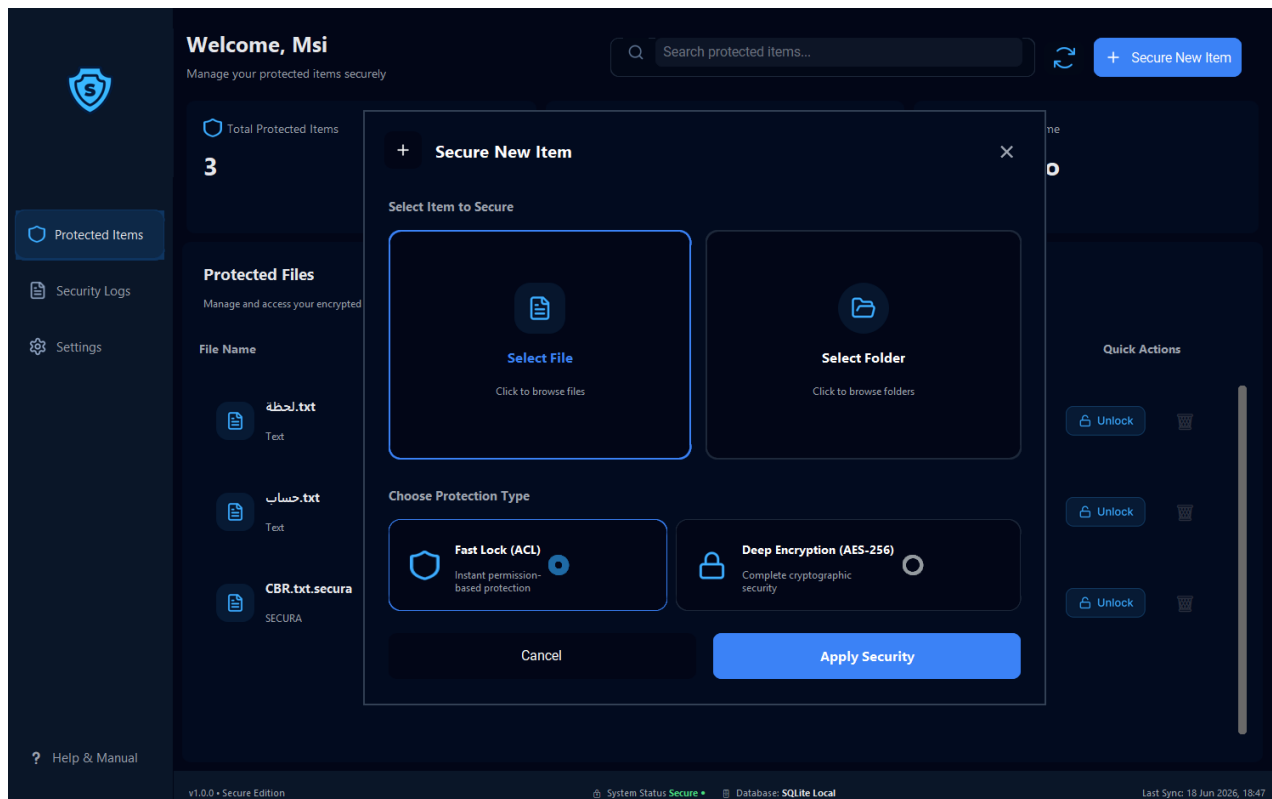
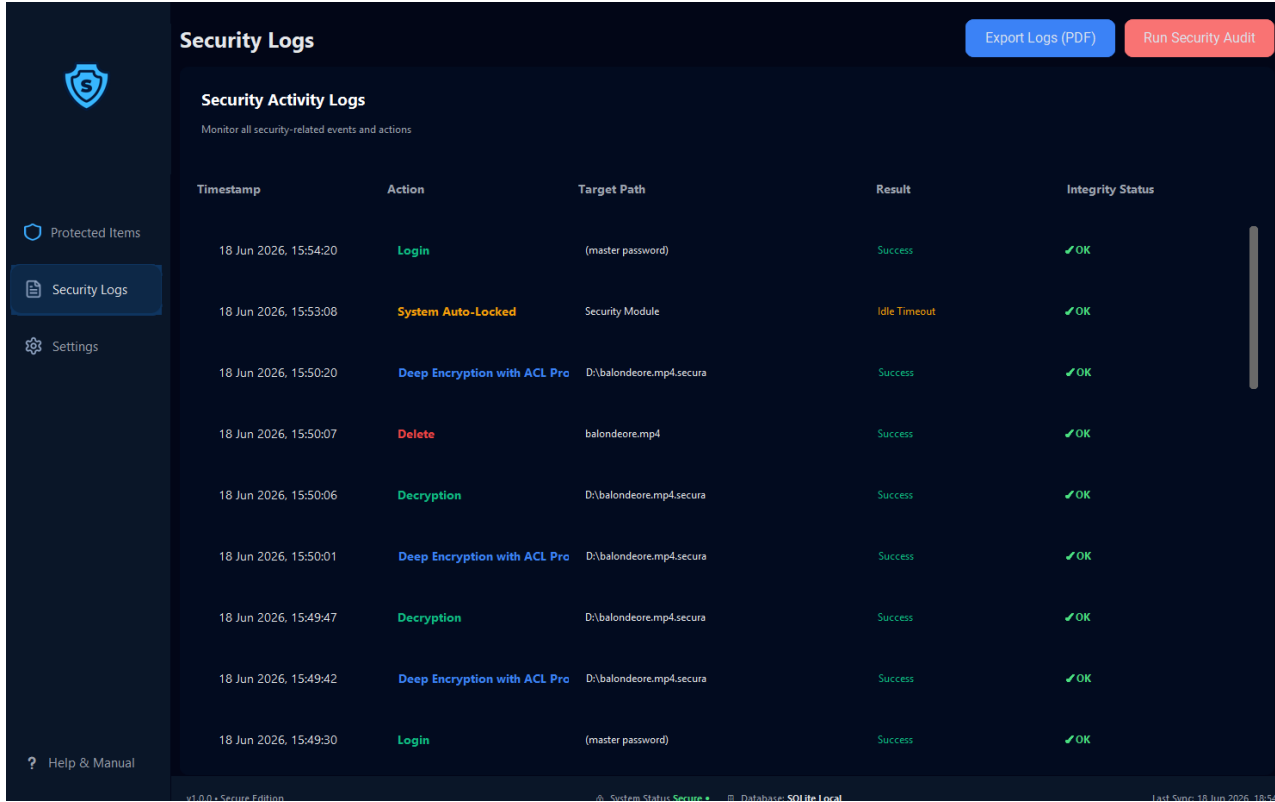


Figure 4.5.4: Secure New Item Interface

B.5 Security Logs

The Security Logs module records authentication attempts, protection operations, recovery actions, and other security-related events.



The screenshot displays the 'Security Logs' interface. On the left is a sidebar with a shield icon and three menu items: 'Protected Items', 'Security Logs' (which is highlighted), and 'Settings'. At the bottom of the sidebar is a 'Help & Manual' link. The main content area is titled 'Security Logs' and includes two buttons: 'Export Logs (PDF)' and 'Run Security Audit'. Below the title is a subtitle 'Security Activity Logs' and a description 'Monitor all security-related events and actions'. A table follows, with columns for 'Timestamp', 'Action', 'Target Path', 'Result', and 'Integrity Status'. The table contains ten rows of log entries. The actions include 'Login', 'System Auto-Locked', 'Deep Encryption with ACL Pro', 'Delete', and 'Decryption'. The results are either 'Success' or 'Idle Timeout', and the integrity status for all entries is 'OK'.

Timestamp	Action	Target Path	Result	Integrity Status
18 Jun 2026, 15:54:20	Login	(master password)	Success	✓OK
18 Jun 2026, 15:53:08	System Auto-Locked	Security Module	Idle Timeout	✓OK
18 Jun 2026, 15:50:20	Deep Encryption with ACL Pro	D:\balondeore.mp4.secura	Success	✓OK
18 Jun 2026, 15:50:07	Delete	balondeore.mp4	Success	✓OK
18 Jun 2026, 15:50:06	Decryption	D:\balondeore.mp4.secura	Success	✓OK
18 Jun 2026, 15:50:01	Deep Encryption with ACL Pro	D:\balondeore.mp4.secura	Success	✓OK
18 Jun 2026, 15:49:47	Decryption	D:\balondeore.mp4.secura	Success	✓OK
18 Jun 2026, 15:49:42	Deep Encryption with ACL Pro	D:\balondeore.mp4.secura	Success	✓OK
18 Jun 2026, 15:49:30	Login	(master password)	Success	✓OK

Figure 4.5.5: Security Logs Interface

Figure 4.5.5 presents the security monitoring interface used to review system events and audit records.

B.6 Cloud Backup, Disaster Recovery and Repair Files

Secura supports cloud backup and disaster recovery through Google Drive integration. Users can synchronize vault information and restore their data when required.

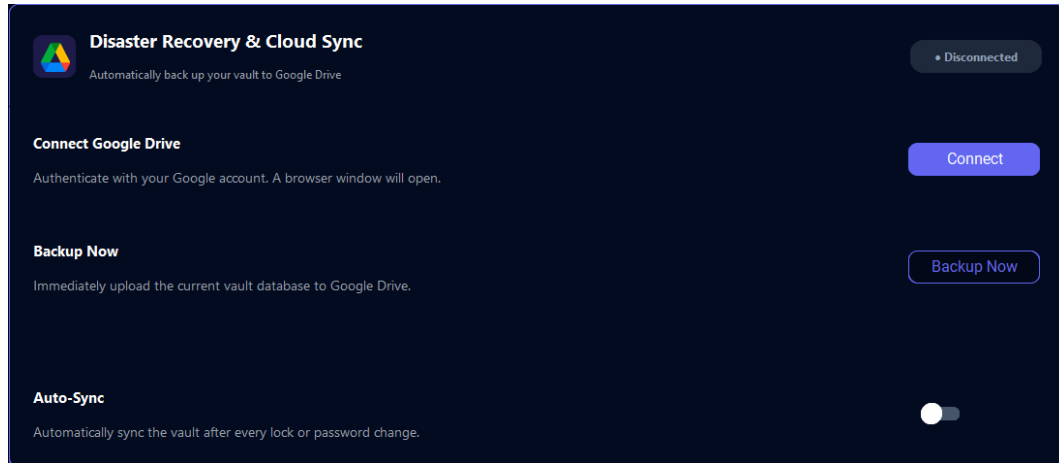


Figure 4.5.6: Cloud Backup Interface

Figure 4.5.6 presents the Cloud Backup interface. The screen allows users to connect their Google Drive account, create manual backups, and enable automatic synchronization for secure vault recovery.

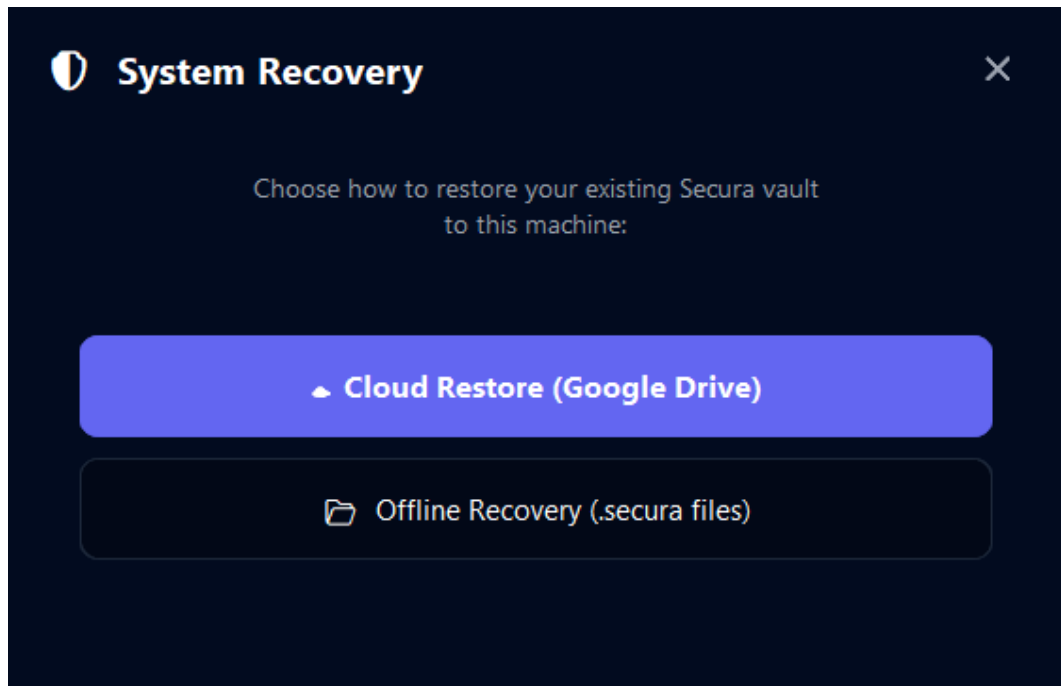
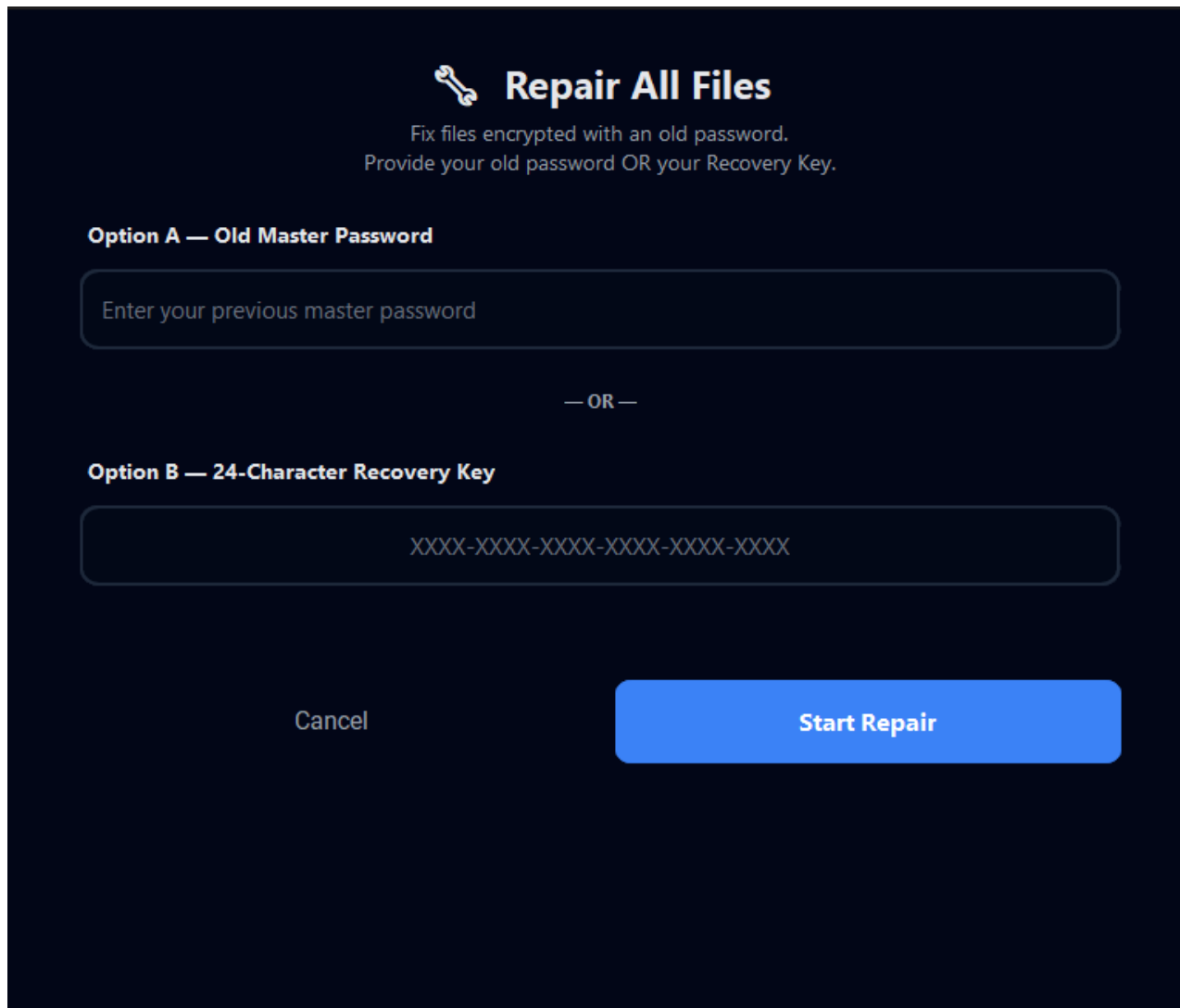


Figure 4.5.7: Disaster Recovery Interface

Figure 4.5.8 presents the System Recovery interface. The screen allows users to restore their Secura vault either from a Google Drive cloud backup or from local recovery files.



The image shows a dark-themed dialog box titled "Repair All Files" with a key icon. Below the title, it says "Fix files encrypted with an old password. Provide your old password OR your Recovery Key." There are two options: "Option A — Old Master Password" with a text input field containing the placeholder "Enter your previous master password", and "Option B — 24-Character Recovery Key" with a text input field containing the placeholder "XXXX-XXXX-XXXX-XXXX-XXXX-XXXX". Between the two options is a separator "— OR —". At the bottom, there are two buttons: "Cancel" and "Start Repair".

Repair All Files

Fix files encrypted with an old password.
Provide your old password OR your Recovery Key.

Option A — Old Master Password

Enter your previous master password

— OR —

Option B — 24-Character Recovery Key

XXXX-XXXX-XXXX-XXXX-XXXX-XXXX

Cancel Start Repair

Figure 4.5.8: Repair Files

Figure 4.5.8 shows the Repair All Files interface. The screen allows users to repair protected files using either the old master password or the recovery key.

B.7 Settings and Maintenance

The Settings interface allows users to configure system behavior, security preferences, and maintenance operations.

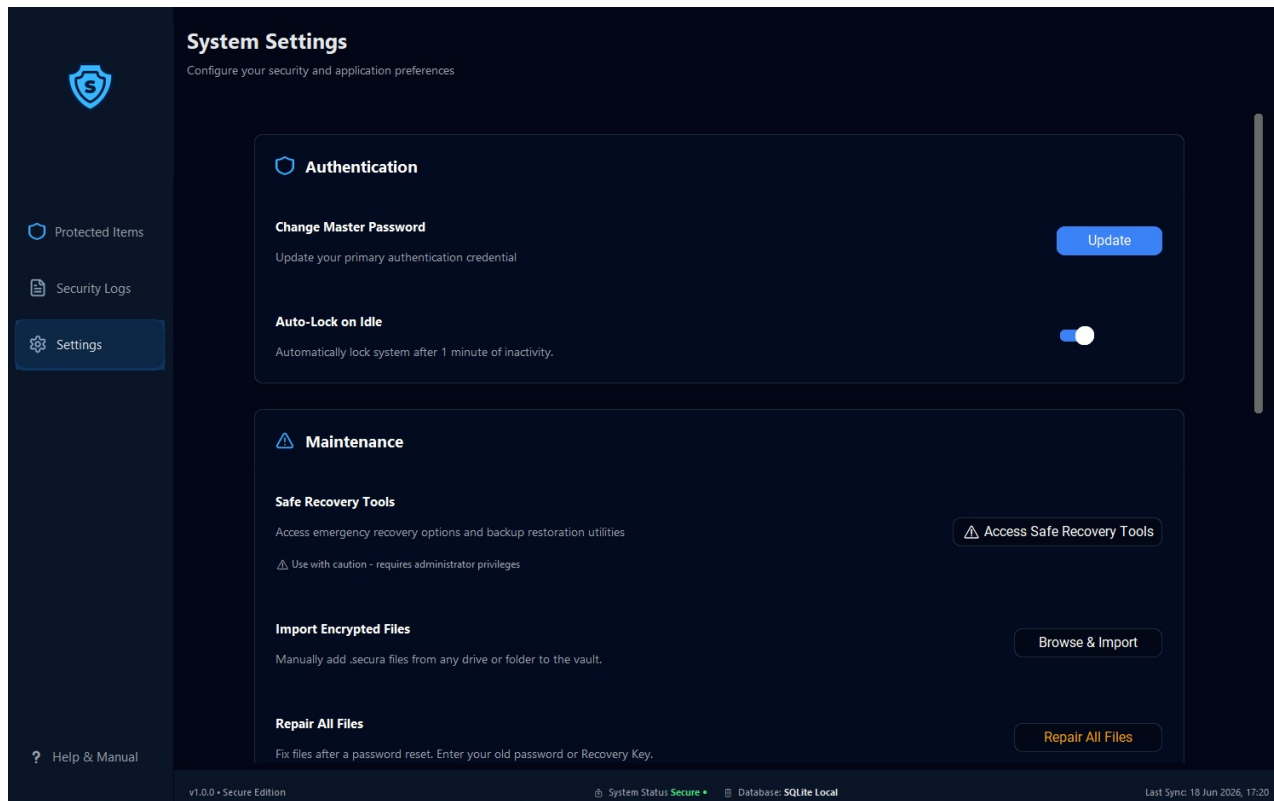


Figure 4.5.9: Settings Interface

Figure 4.5.9 presents the settings panel used to manage application preferences and maintenance functions.

B.8 Help and User Manual

The integrated help system provides guidance on system functionality, recovery procedures, and operational instructions.

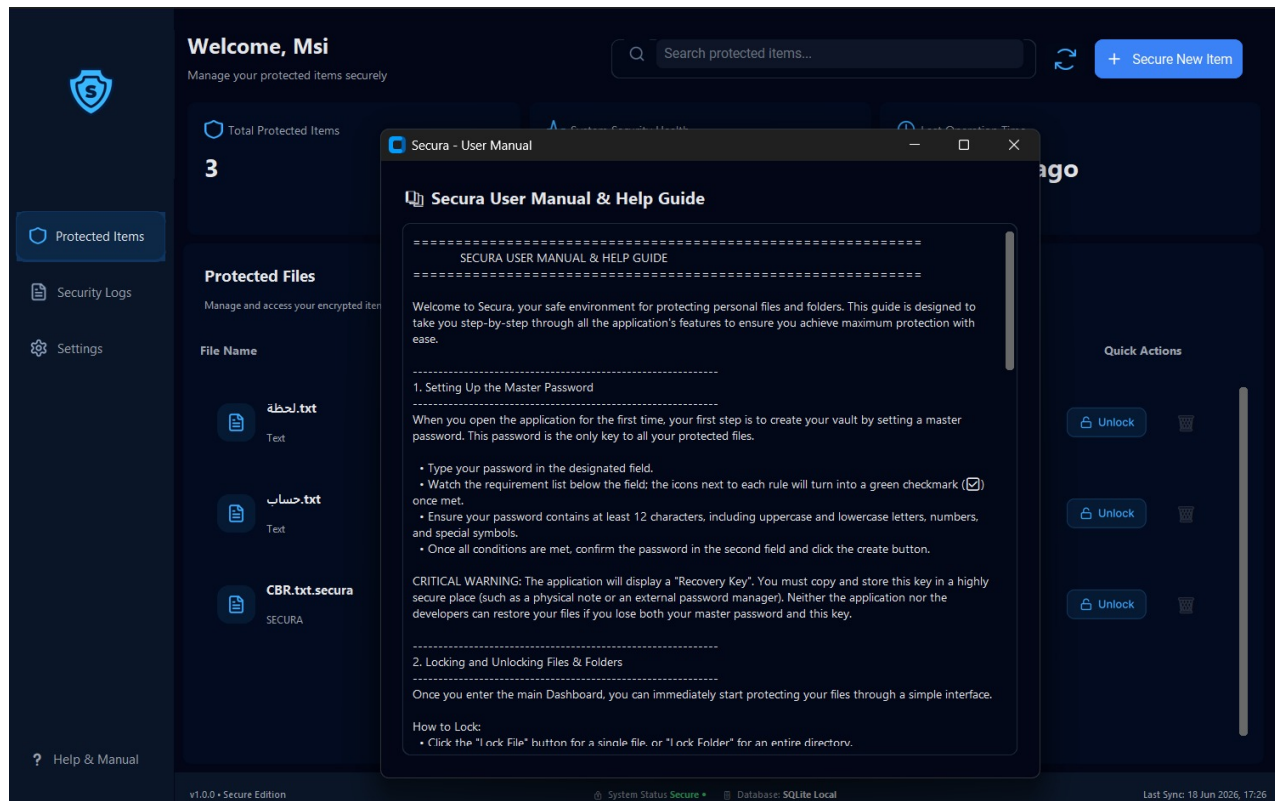


Figure 4.5.10: Help and Manual Interface

Figure 4.5.10 shows the built-in help and documentation module available to users.